

A Taste of Categorical Semantics

Marco Paviotti

March 19, 2025

Contents

1	Introduction	2
2	Elements of Set Theory	4
2.1	Relations and Functions	4
2.2	Cardinality and Isomorphism	5
2.3	Indexed Families of Sets	5
2.4	The Russel's Paradox	6
2.4.1	The Axiom of Regularity (Foundation)	6
2.5	Exercises	7
3	Categories	8
3.1	Initial and terminal objects	9
3.2	The Natural Numbers object	10
3.3	Isomorphisms	11
3.4	Constructions on Categories	11
3.4.1	Product categories	11
3.4.2	Opposite categories	12
3.4.3	Arrow Categories	12
3.4.4	Slice Categories	12
3.5	Products, CoProducts and Exponentials	13
3.5.1	Products	13
3.5.2	Coproducts	14
3.5.3	Exponentials	16
3.6	Exercises	17
4	Semantics of the Simply Typed λ-Calculus	18
4.1	Syntax	18
4.2	Semantics	19
4.3	Exercises	20

5	Functors and Natural Transformations	21
5.1	Functors	21
5.2	Natural Transformations	23
5.3	The Homset Functor	23
5.4	The Yoneda Lemma	25
5.5	The Category of Categories	26
5.6	Equivalence of Categories	27
5.7	Exercises	27
6	Limits and Colimits	27
7	Algebraic Data Types	28
8	Adjunctions	29
8.0.1	Adjunctions	29
8.1	Instances of Adjunctions	30
8.1.1	Initial and Terminal Objects	30
8.1.2	(Co)products and Products	30
8.1.3	Exponentials	30
8.1.4	(Co)Limits	31
9	Semantics of Predicative Polymorphism	31
9.0.1	Syntax	32
9.0.2	Semantics	34
10	The Call-By-Push-Value λ-calculus	36
11	Monads	36
11.0.1	Adjunctions determine Monads	36
11.1	The Kleisli Category	37
11.2	The Computational λ -calculus	37
11.2.1	Syntax	37
11.2.2	Semantics	38

1 Introduction

When learning an abstract mathematical concept, it is helpful to have a concrete notion of the subject being studied; without this, the abstraction may lack meaningful context. There are probably two approaches to learning category theory in computer science: through the lens of mathematics or through that of functional programming. While the mathematical perspective is arguably the most rigorous, many computer scientists may find the functional programming perspective more accessible. Indeed, some authors have already chosen to adopt this path [5].

In these notes, we take a different approach – one which lies between mathematics and programming languages. This approach aligns with the mathematical treatment of programming languages known as *denotational semantics*. The idealized concept in

this context is that a *type* can be viewed as a *set*, and a *program* as a *function* between sets. However, as programming languages incorporate more features, maintaining this simplistic view becomes increasingly challenging. By employing category theory, we generalize this perspective: a type corresponds to an *object*, and a program corresponds to an *arrow*. With this idea in mind, we will model the simply typed λ -calculus, predicative polymorphism and λ -calculi with computational effects.

For the interested reader who wants to dive deeper in these subjects, there is a plethora of very well-written books about category for a more in-depth introduction on the subject [1, 4].

In these notes, we will try to convince the reader that category theory covers a wide variety of subjects; from logic to type theory, hence programming, algebra and topology and it is therefore a subject worth studying for the reader's own good.

To give a taste for why this is true let us look at three most common axioms which we can find in logic, type theory and algebra. In particular, reflexivity and transitivity. Reflexivity states that every proposition A entails itself, written $A \vdash A$ in logic. In type theory, this corresponds to the *variable rule*, which states that if a variable x has type A , written $x : A$, then we can type check the program $x : A$. Similarly, transitivity ensures that proofs of propositions can be chained: if A entails B and B entails C then A entails C . This is similar to what happens in type theory: if t is a program of type B with an open variable of type A and t' is a program of type C open in $y : B$ then substituting t for y in t' yields a program of type C open in $x : A$. In algebra, a *preorder* has exactly the same properties as for all A in a preorder we have $A \leq A$ and for all A, B, C , if $A \leq B$ and $B \leq C$ then obviously $A \leq C$. The table below summarises the concepts we explained so far:

Logic	Type Theory	Algebra
$A \vdash A$	$x : A \vdash x : A$	$A \leq A$
$\frac{A \vdash B \quad B \vdash C}{A \vdash C}$	$\frac{x : A \vdash t : B \quad y : B \vdash t' : C}{x : A \vdash \{t/y\}t' : C}$	$\frac{A \leq B \quad B \leq C}{A \leq C}$

It should now be evident that three seemingly disconnected areas of mathematics – logic, computer science, and algebra – are deeply interconnected. This connection has been termed *computational trinitarianism*, the idea that logic, type theory, and algebra are three perspectives on the same underlying objects. This perspective reinforces the thesis that computability is an inherent concept in the foundations of mathematics.

In 1945, Saunders MacLane and Samuel Eilenberg, while working on algebraic topology, observed recurring patterns in algebra that could be generalized. He developed an abstract axiomatization encompassing many aspects of algebra, which led to the formulation of the *axioms of a category* [4]. A category consists of objects and morphisms (arrows) between them, governed by two fundamental axioms. The first, *identity*, reflects reflexivity by requiring an identity morphism $\text{id}_A : A \rightarrow A$ for every object. The second, *composition*, embodies transitivity by allowing morphisms $f : A \rightarrow B$ and $g : B \rightarrow C$ to compose into $g \circ f : A \rightarrow C$.

$$A \xrightarrow{\text{id}_A} A \quad A \xrightarrow{f} B \xrightarrow{g} C \quad A \xrightarrow{g \circ f} C$$

The reader is encouraged to keep these analogies in mind throughout this article, as they provide intuitive insights into the abstract categorical concepts we will explore.

Lastly, since category theory is formulated on top of set theory, we will briefly summarise the basic notions of sets, size, families of sets, and Russell’s Paradox in the next section. The Russell’s Paradox, in particular, is a pivotal result in naive set theory, offering insights into the design choices behind category theory and highlighting certain challenges related to polymorphism in programming languages.

2 Elements of Set Theory

A *set* (or class) is an unordered collection of objects called *elements*. If an object a is an element of a set X , we write $a \in X$ and say “ a belongs to X .”

One of the most fundamental sets is the *empty set*, denoted $\emptyset$, which contains no elements. It is important to distinguish between $\emptyset$ and the *singleton set* $\{\emptyset\}$, which contains the empty set as its only element.

Other notable sets include the set of *natural numbers*, $\mathbb{N} = \{1, 2, 3, \dots\}$, and the set of *natural numbers with zero*, $\mathbb{N}_0 = \{0, 1, 2, 3, \dots\}$.

Given two sets A and B , we can construct their *Cartesian product*, $A \times B$, which is the set of all ordered pairs (a, b) such that $a \in A$ and $b \in B$. Similarly, the *union* of two sets A and B , written $A \cup B$, is the set containing all elements of A and B , with duplicates removed. A related concept is the *disjoint union*, $A \uplus B$, which pairs elements with tags to distinguish their origin: $(1, a)$ for $a \in A$ and $(2, b)$ for $b \in B$. Given an equivalence relation $\sim$ on X , the *quotient* of a set X by an equivalence relation $\sim$ is the set of equivalence classes:

$$X/\sim = \{[x] \mid x \in X\},$$

where $[x] = \{y \in X \mid x \sim y\}$. The quotient map $q : X \rightarrow X/\sim$ sends each element $x \in X$ to its equivalence class $[x]$. The quotient set partitions X into disjoint equivalence classes. The union of two sets A and B can be expressed as a quotiented disjoint union:

$$A \cup B = (A \uplus B) / \sim,$$

where $(1, a) \sim (2, b)$ if and only if $a = b$.

2.1 Relations and Functions

A *relation* $R \subseteq A \times B$ is a subset of the Cartesian product $A \times B$, linking certain elements of A to elements of B . A *function* is a special type of relation $f \subseteq A \times B$ such that for every $a \in A$, there exists exactly one $b \in B$ related to it. The set of all functions from A to B is denoted $A \rightarrow B$.

Two special cases of functions are worth noting. First, there is only one function from the empty set $\emptyset$ to any set A , which is the empty relation $! \subseteq \emptyset \times A$. Second, there is only one function from any set A to a singleton set $\{*\}$, which maps every element $a \in A$ to $*$. This function is also denoted $!$, and its meaning is typically clear from context.

An *equivalence relation* on a set X is a relation R that satisfies three properties: *reflexivity*, meaning that every element is related to itself ($\forall x \in X, xRx$); *symmetry*, meaning that if one element is related to another, then the second is related to the first ($\forall x, y \in X, xRy \Rightarrow yRx$); and *transitivity*, meaning that if one element is related to a second and the second to a third, then the first must be related to the third ($\forall x, y, z \in X, xRy$ and $yRz \Rightarrow xRz$).

A *preorder* is a relation that satisfies only reflexivity and transitivity, making it a weaker form of ordering that allows for indistinguishable elements. A *partial order* strengthens this by also requiring *antisymmetry*, which states that if two elements are mutually related, they must be equal ($\forall x, y \in X, xRy$ and $yRx \Rightarrow x = y$).

2.2 Cardinality and Isomorphism

For two sets X, Y , a function $f : X \rightarrow Y$ is called *injective* (or one-to-one) if different inputs always produce different outputs, formally expressed as $f(x_1) = f(x_2) \Rightarrow x_1 = x_2$. It is *surjective* (or onto) if every element of Y is mapped to by at least one element of X , meaning $\forall y \in Y, \exists x \in X$ such that $f(x) = y$. When a function is both injective and surjective, it is called a *bijection* (or a one-to-one correspondence), ensuring that every element of X is uniquely paired with an element of Y and vice versa, allowing for the existence of an inverse function $f^{-1} : Y \rightarrow X$.

The *size* or *cardinality* of a set measures how many elements it contains. Two sets are said to have the same cardinality, or to be *isomorphic*, if there exists a bijective function $f : A \rightarrow B$ with an inverse $f^{-1} : B \rightarrow A$ such that $f(f^{-1}(x)) = x$ and $f^{-1}(f(x)) = x$ for all x .

A set A is *finite* if it is isomorphic to the set $\{m \in \mathbb{N} \mid m \leq n\}$ for some $n \in \mathbb{N}$. In this case, we can enumerate its elements as $A = \{a_1, a_2, \dots, a_n\}$. A set is *infinite* if it is not finite. A set is *countable* if it is isomorphic to the natural numbers $\mathbb{N}$.

2.3 Indexed Families of Sets

For a set I , an *indexed family of sets* is a collection of sets $\{A_i\}_{i \in I}$, where each set A_i is associated with an index $i \in I$. If I is finite, we can write the union and Cartesian product of these sets as:

$$A_1 \cup A_2 \cup \dots \cup A_n \quad \text{and} \quad A_1 \times A_2 \times \dots \times A_n.$$

If I is infinite, the *union* of the family is:

$$\bigcup_{i \in I} A_i = \{a \in A_i \mid i \in I\},$$

and the *dependent product* (or *infinite product*) is the set of functions $f : I \rightarrow \bigcup_{i \in I} A_i$ such that $f(i) \in A_i$ for all $i \in I$:

$$\prod_{i \in I} A_i = \{f : I \rightarrow \bigcup_{i \in I} A_i \mid f(i) \in A_i\}.$$

2.4 The Russel's Paradox

Russell's Paradox is a fundamental problem in set theory, discovered by Bertrand Russell in 1901. It reveals a contradiction in naive set theory, which allowed the formation of any set based on a defining property, without restrictions. The paradox shows that such a theory can lead to logical inconsistencies.

The paradox arises when we consider the set of all sets that do not contain themselves as a member. Let's define this set as R . Formally, R is the set of all sets that do not contain themselves as a member. In other words:

$$R = \{x \mid x \notin x\}$$

Here, R is the set of all sets x such that x does *not* contain itself as a member.

Now, the central question of the paradox is: **Does the set R contain itself?**

To answer this, we explore two possibilities. The case when $R \in R$ and the case when $R \notin R$. If R is a member of itself, then by the definition of R , it must not contain itself (because R is the set of all sets that do not contain themselves). Therefore, if $R \in R$, it must follow that $R \notin R$, which is a contradiction. If R is *not* a member of itself, then by the definition of R , it must contain itself (because R is the set of all sets that do not contain themselves, and R would be one of those sets). Therefore, if $R \notin R$, it must follow that $R \in R$, which is also a contradiction.

This contradiction shows that the assumption that such a set R can exist leads to an inconsistency. The paradox demonstrates that naive set theory, which allowed for the creation of sets like R , is inherently flawed.

2.4.1 The Axiom of Regularity (Foundation)

The Axiom of Regularity, also known as the Axiom of Foundation, is one of the axioms in Zermelo-Fraenkel set theory (ZF), designed to prevent certain paradoxes like Russell's Paradox.

The Axiom of Regularity states:

$$\forall A (A \neq \emptyset \Rightarrow \exists x \in A (x \cap A = \emptyset))$$

This means that for every non-empty set A , there exists an element $x \in A$ such that x and A are disjoint sets.

The Axiom of Regularity essentially says that *no set can be a member of itself* (directly or indirectly), and it prevents sets from containing themselves or forming cycles. In other words, it ensures that sets are well-founded, meaning they cannot "loop back" on themselves in any way.

In the case of Russell's Paradox, we defined a set R as:

$$R = \{x \mid x \notin x\}$$

The paradox arose because the set R seemed to both contain itself and not contain itself, depending on the assumption. The Axiom of Regularity helps avoid such contradictions by ensuring that no set can be a member of itself. It guarantees that sets like R , which would allow self-referencing and circular definitions, cannot exist.

Let A be a set, and apply the axiom of regularity to the singleton set containing A , that is $\{A\}$. By the axiom of regularity there must be an element of A which is disjoint from $\{A\}$. Since A is the only element of $\{A\}$, it must be that A is disjoint from $\{A\}$. Therefore, since $A \cap \{A\} = \emptyset$ that means that A does not contain itself by definition of intersection.

2.5 Exercises

1. Define the set $A = \{x \in \mathbb{Z} \mid x^2 < 25\}$ explicitly by listing all its elements.
2. Consider the set $B = \{(x, y) \in \mathbb{Z}^2 \mid x + y = 5\}$. List five elements of B .
3. Let R be a relation on $\mathbb{Z}$ defined by xRy if and only if $x - y$ is even. Prove that R is an equivalence relation.
4. Define a relation S on $\mathbb{R}$ by xSy if and only if $xy \geq 0$. Determine whether S is reflexive, symmetric, and/or transitive.
5. Consider the relation T on the power set $\mathcal{P}(\mathbb{N})$ defined by ATB if and only if $A \subseteq B$. Is T a partial order?
6. Define a function $f : \mathbb{Z} \rightarrow \mathbb{Z}$ by $f(x) = x^2 - 3x + 2$. Determine whether f is injective and/or surjective.
7. Consider the function $g : \mathbb{Q} \rightarrow \mathbb{Q}$ defined by $g(x) = 2x + 1$. Show that g is a bijection and find its inverse.
8. Let $h : \mathbb{N} \rightarrow \mathbb{N}$ be defined by

$$h(n) = \begin{cases} n/2, & \text{if } n \text{ is even} \\ 3n + 1, & \text{if } n \text{ is odd} \end{cases}$$

Compute $h(1), h(2), h(3), h(4), h(5)$.

9. Prove that the interval $(0, 1)$ and $(0, 2)$ in $\mathbb{R}$ have the same cardinality.
10. Show that $\mathbb{N}$ and $\mathbb{Z}$ have the same cardinality by constructing an explicit bijection.
11. Prove that the set of all finite binary strings is countable, while the set of all infinite binary sequences is uncountable.
12. Let $A_n = \{x \in \mathbb{R} \mid x > \frac{1}{n}\}$ for $n \in \mathbb{N}$. Find $\bigcap_{n=1}^{\infty} A_n$.

13. Consider the sequence of sets $B_n = \{0, 1, 2, \dots, n\}$. Determine $\bigcup_{n=1}^{\infty} B_n$ and $\bigcap_{n=1}^{\infty} B_n$.
14. Compute the Cartesian product

$$\prod_{i=1}^{\infty} \{0, 1\}$$

and determine whether it is countable or uncountable.

3 Categories

As explained above, the intuition the reader should have is that when talking about well-typed programming languages, types should be regarded as *objects* and programs should be regarded as *arrows*. This sort of motivates the definition of a category:

Definition 3.1 (Category). *A category C is a collection of objects $A, B, C \dots$ denoted by $\text{Obj}(C)$ and a set of arrows $f, g, h \dots$ denoted by $\text{Arr}(C)$. Additionally, for each object A there exists an identity arrow $\text{id}_A : A \rightarrow A$ such that and for arrows $f : A \rightarrow B$ and $g : B \rightarrow C$ we there exists an arrow $g \circ f : A \rightarrow C$. We picture these as mentioned in the introduction:*

$$A \xrightarrow{\text{id}_A} A \quad A \xrightarrow{f} B \xrightarrow{g} C$$

Additionally, these arrows obey the identity and associativity laws:

$$\begin{aligned}\text{id}_A \circ f &= f \circ \text{id}_A = f \\ f \circ (g \circ h) &= (f \circ g) \circ h\end{aligned}$$

Examples of categories are the category of sets, denoted by **Set**, where objects are sets and arrows are functions, the category of sets and relations **Rel**, the category of partial order sets **Pos** and monotone functions, the category of groups **Grp** of groups and group homomorphisms, the category of topological spaces **Top** of topological spaces and continuous functions between them.

To avoid clutter, we simply write $X \in \mathcal{C}$ for an object in a category $\mathcal{C}$ and $f \in \mathcal{C}$ for a morphism in a category $\mathcal{C}$. For objects $X, Y \in \mathcal{C}$ we write $\mathcal{C}(X, Y)$ for the collection of morphisms from X to Y which we call the homset.

The fact that a category is defined in terms of collections objects and morphisms is to avoid paradoxes such as the one in Section 2.4. For example, the category **Set** is too big for the objects to form a set, since the set of sets does not exist. When the collection of objects is a set we say the category is *small*. Similarly when the homset is a set we say the category is *locally small*.

3.1 Initial and terminal objects

Consider the false proposition in logic. When we assume something absurd we can derive anything, hence a false proposition entails any formula A . In programming this is expressed in terms of empty data types which are denoted sometimes with 0 . Having a piece of data of type 0 , say $x : 0$, is absurd since the empty type contains nothing, thus performing case analysis on this data will be vacuous as state in the axiom below

$$\overline{x : 0 \vdash \text{case } x \text{ of } \{\} : A}$$

The axiom above may look strange at first. The variable x is case analysed on but there is no term inside the case operator. But this makes sense because case analysis on a data structure means providing a program to continue with for each constructor in the data structure, however there is no constructor in the empty type so there is nothing further we need to provide.

In category theory this is expressed in terms of initial objects. The initial object is an object denoted by 0 such that for every other object A there exist a unique arrow between 0 and A . We draw a dashed arrow as follows to indicate the arrow is unique:

$$0 \dashrightarrow^! A$$

In other words, there is a unique proof which takes a proof of the false statement and returns a proof of any statement A . This can also be interpreted in programming as the program from the empty type into any type. Intuitively, the only way to produce something of type A is to case analyse on the empty type, but because this type does not contain anything the program has nothing further to compute and the case is vacuous.

Similarly, the *terminal* object 1 represent the true statement or the type with only one element in it, the *unit type*. In programming terms, given any input of type A there exists exactly one program producing an element of the unit type, that is the program which discards the input and returns the single element in the unit type. Categorically, and it is such that for every object A there is a unique arrow into the terminal object:

$$A \dashrightarrow^! 1$$

It is important to know that such objects in category theory are unique only up-to isomorphism.

In **Set**, the empty set $\emptyset$ is the initial object since there is a unique function from the empty set to any other set A , that is the empty function or empty relation. Similarly, the terminal object is any set containing exactly one element, the singleton set. Consider the set $\{*\}$. For any other given set A there is a unique function into $\{*\}$ which is the constant function mapping every element $x \in A$ into $\{*\}$. Notice that there are many terminal objects in **Set**, but they are all isomorphic in that they all contain only one element. Also **Set** is a special category of sorts, in fact, it enjoys the property that the set of morphisms from 1 to any set A is isomorphic to A itself since any function $1 \xrightarrow{x} A$ can map into exactly one element in A .

3.2 The Natural Numbers object

The *natural numbers object* (NNO) is an abstract representation of the natural numbers $\mathbb{N}$ within a category. It consists of an object $\mathbb{N}$ along with two morphisms: the zero morphism $1 \xrightarrow{\text{zero}} \mathbb{N}$ and the successor morphism $\mathbb{N} \xrightarrow{\text{succ}} \mathbb{N}$. These morphisms encode the structure of the natural numbers, with zero corresponding to the base element 0 and succ representing the successor function, which maps a number n to $n + 1$.

The NNO is characterized by a universal property: for any object X in the category and any pair of morphisms $1 \xrightarrow{f} X$ (representing a base case) and $X \xrightarrow{g} X$ (representing a recursive step), there exists a unique morphism $\mathbb{N} \xrightarrow{([f, g])} X$ such that $([f, g]) \circ \text{zero} = f$ and $([f, g]) \circ \text{succ} = g \circ ([f, g])$.

$$\begin{array}{ccccc} 1 & \xrightarrow{\text{zero}} & \mathbb{N} & \xrightarrow{\text{succ}} & \mathbb{N} \\ & \searrow f & \downarrow \text{([f, g])} & & \downarrow \text{([f, g])} \\ & & X & \xrightarrow{g} & X \end{array}$$

What is important to notice is that the existence of this map provides a way of defining functions by recursion over the natural numbers, while uniqueness gives an *inductive proof principle*.

As an example, in **Set**, we can prove that all natural numbers are either even or odd by defining a subset of the natural numbers

$$P = \{n \in \mathbb{N} \mid \text{even}(n) \vee \text{odd}(n)\}$$

We aim to show that $P \cong \mathbb{N}$, i.e., every natural number is either even or odd. To achieve this, we construct the necessary maps that induce a unique morphism from $\mathbb{N}$ to P , which establishes the required property.

First, we define a base case function $b : 1 \rightarrow P$ by setting $b(0) = 0$, which is even and thus belongs to P . Next, we define an induction step function $i : P \rightarrow P$ by specifying that if $n \in P$, then $i(n) = n + 1$ must also be in P . We must prove that this mapping is well-defined, that is that $n + 1$ is either even or odd. This is the crux of the proof since it corresponds to our inductive step. Obviously, if n is even then $n + 1$ is odd thus it belongs to P and similarly for when n is odd. Notice we have defined an object P along with two mappings i and b such that the following diagrams commute:

$$\begin{array}{ccccc} & & P & \xrightarrow{i} & P \\ & \nearrow b & \downarrow \subseteq & & \downarrow \subseteq \\ 1 & \xrightarrow{\text{zero}} & \mathbb{N} & \xrightarrow{\text{succ}} & \mathbb{N} \end{array}$$

By the universal property of the natural numbers, there exists a unique map $([b, i]) : \mathbb{N} \rightarrow P$ satisfying the recursive definition given by b and i . It is easy now to prove that $P \cong \mathbb{N}$ using set inclusion by showing the maps $P \subseteq \mathbb{N}$ and $([b, i]) : \mathbb{N} \rightarrow P$ are mutually inverses to each other, meaning that

$$([b, i]) \circ \subseteq = \text{id}_P \quad \subseteq \circ ([b, i]) = \text{id}_{\mathbb{N}}$$

We leave this last step as an exercise.

3.3 Isomorphisms

Isomorphism is a fundamental concept that captures the idea of two objects being “essentially the same”. An isomorphism between two objects indicates that they are structurally identical, even if they might appear different externally. While the concept of “essentially the same” is central to isomorphism, it is not always straightforward to understand what this means in different settings. An isomorphism in **Set** is a pair of functions which are inverses to each other. This corresponds to saying that two sets are isomorphic if they have the same cardinality, which is equivalent to saying that there exists a function $f : A \rightarrow B$ such that is surjective and injective.

However, consider the the category **Pos** of partial order sets and order preserving functions. The following are two posets which are not isomorphic:



since in the right-hand side poset the $\perp$ element is not ordered with 1. Despite the fact that these two posets are in bijection they are not isomorphic because any bijection would be able to preserve the order $\perp \leq 1$ from the left to the right-hand side poset (while still being a bijection).

Category theory abstracts the notion of isomorphism by stating that two objects are isomorphic if and only if there exists a pair of morphisms which are inverse to each other

Definition 3.2 (Isomorphism). *Two given objects A and B are isomorphic, written $A \cong B$ iff there exists an arrow $f : A \rightarrow B$ that has an inverse $g : B \rightarrow A$ such that*

$$g \circ f = id_A \quad f \circ g = id_B$$

This definition depends of course on the definition of morphism, in particular, an isomorphism is defined by what the arrows look like and by what we can observe through them rather than what are the objects themselves.

3.4 Constructions on Categories

3.4.1 Product categories

A *product category* refers to the *Cartesian product* of two categories. Given two categories $\mathcal{C}$ and $\mathcal{D}$, their product category, denoted as $\mathcal{C} \times \mathcal{D}$, is a category where objects are pairs (C, D) , where C is an object from $\mathcal{C}$ and D is an object from $\mathcal{D}$.

Morphisms are pairs (f, g) , where $f : C \rightarrow C'$ is a morphism in $\mathcal{C}$, and $g : D \rightarrow D'$ is a morphism in $\mathcal{D}$.

Composition of morphisms is defined component-wise as

$$(f', g') \circ (f, g) = (f' \circ f, g' \circ g).$$

Identity morphisms are given by (id_C, id_D) , where id_C and id_D are identity morphisms in $\mathcal{C}$ and $\mathcal{D}$, respectively.

Intuitively, a *product category* captures the structure of both C and D simultaneously, treating them as independent but parallel components.

3.4.2 Opposite categories

An *opposite category* refers to a way of reversing the structure of a given category. If we have a category C , its opposite category, denoted C^{op} , is formed by reversing the direction of all morphisms while keeping the same objects.

Definition 3.3 (Opposite category). *For a category C , there is a category C^{op} which has as objects the same objects as C and for every morphism $f : A \rightarrow B$ in C a morphism $f^{\text{op}} : B \rightarrow A$*

$$f \in C(X, Y) \iff f^{\text{op}} \in C^{\text{op}}(Y, X)$$

In other words, for every morphism $f : A \rightarrow B$ in C , there is a corresponding morphism $f^{\text{op}} : B \rightarrow A$ in C^{op} . The composition of morphisms in C^{op} is defined in reverse order compared to C , meaning that if $f : A \rightarrow B$ and $g : B \rightarrow C$ are morphisms in C , their composition in C^{op} is given by $g^{\text{op}} \circ f^{\text{op}}$, corresponding to $(f \circ g)^{\text{op}}$ in C .

The opposite category is a useful tool for exploring dualities, where a statement about C has a corresponding dual statement about C^{op} . We will go back to this topic when we introduce the concept of a functor in Section 5.

3.4.3 Arrow Categories

The *arrow category* of a category C , denoted as $C^{\rightarrow}$, is the category where objects are the morphisms of C . That is, an object in $C^{\rightarrow}$ is a morphism $f : A \rightarrow B$ from C .

Morphisms between two objects $f : A \rightarrow B$ and $g : C \rightarrow D$ are pairs of morphisms $(\alpha : A \rightarrow C, \beta : B \rightarrow D)$ in C such that the following commutative square holds:

$$\begin{array}{ccc} A & \xrightarrow{f} & B \\ \downarrow \alpha & & \downarrow \beta \\ C & \xrightarrow{g} & D \end{array}$$

That is, $g \circ \alpha = \beta \circ f$.

Composition is defined component-wise: if (α, β) is a morphism from f to g , and (α', β') is a morphism from g to h , their composition is $(\alpha' \circ \alpha, \beta' \circ \beta)$.

Identity morphisms are given by $(\text{id}_A, \text{id}_B)$ for each morphism $f : A \rightarrow B$.

The arrow category encodes the relationships between morphisms in C , making it useful for studying *natural transformations*, *functors*, and *factorization properties*.

3.4.4 Slice Categories

The *slice category* of a category C over an object X , denoted as C/X , is a category where objects are morphisms in C with codomain X . That is, an object in C/X is a pair (A, f) , where A is an object in C and $f : A \rightarrow X$ is a morphism in C .

Morphisms between two objects (A, f) and (B, g) are morphisms $h : A \rightarrow B$ in C such that the following commutative triangle holds:

$$\begin{array}{ccc} A & \xrightarrow{h} & B \\ & \searrow f & \swarrow g \\ & X & \end{array}$$

That is, $g \circ h = f$.

Composition is inherited from C , meaning if $h : (A, f) \rightarrow (B, g)$ and $k : (B, g) \rightarrow (C, t)$, their composition is $k \circ h : (A, f) \rightarrow (C, t)$.

Identity morphisms are given by id_A for each object (A, f) .

The slice category C/X captures the *structure of objects over X* and is commonly used in *fibered categories*, *topos theory*, and *universal properties*.

3.5 Products, CoProducts and Exponentials

In this section we are going to take a look at the categorical semantics of the simply typed λ -calculus. This is a λ -calculus with natural numbers, pairs and function types therefore it is only natural that to model these we need their logical and algebraic counterparts.

3.5.1 Products

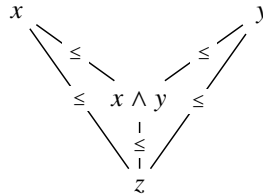
In logic, the *and* operator is introduced by stating that if Γ is a set of true propositions which entails A and this context Γ entails also B then of course $\Gamma \vdash B$. This is called the introduction rule of the product:

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B}$$

The product has also two elimination rules given by:

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} \quad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B}$$

In algebra, a set which possess this structures is called a *lower semilattice*. That is a set X such that for every $a, b \in X$, there exists an element $a \wedge b$ called the *meet* or the *greatest lower bound* of a and b which has the property that for every other lower bound z , that is $z \leq a$ and $z \leq b$ we have that $z \leq a \wedge b$.



We proceed now to generalise the concept of greatest lower bound to the categorical notion of product $A \times B$.

Definition 3.4 (Product). For two objects A and B the product is an object P equipped with two arrows $\pi_1 : P \rightarrow A$ and $\pi_2 : P \rightarrow B$ such that for any object Z with arrows $f : Z \rightarrow A$ and $g : Z \rightarrow B$ there exists a unique morphism $h : Z \rightarrow P$ making the following diagram commute:

$$\begin{array}{ccccc} A & \xleftarrow{\pi_1} & P & \xrightarrow{\pi_2} & B \\ & \nwarrow f & \uparrow h & \nearrow g & \\ & & Z & & \end{array}$$

We write $A \times B$ for the product of two objects A and B and $\langle f, g \rangle$ for h .

Note that this definition corresponds to the notion of product in **Set**

$$A \times B \triangleq \{ \langle a, b \rangle \mid a \in A \text{ and } b \in B \}$$

where the projections π_1 and π_2 are simply the functions discarding one component of the pair $\pi_1(a, b) = a$ and $\pi_2(a, b) = b$ and for every element $1 \xrightarrow{a} A$ and $1 \xrightarrow{b} B$ the pairing is defined by $(a, b) \mapsto \langle a, b \rangle$. Note that the existence condition of the pairing function enforces that every element of A and B are in the product (no noise) and the uniqueness condition ensures that there is no more elements in $A \times B$ than the ones that are coming from A and B (no junk).

Notice also that we could have defined the product in many other ways, but as long as the categorical definition is satisfied, all these definitions are isomorphic. The reader should convince themselves that is true by proving the following proposition:

Proposition 3.1 (Products are unique up to isomorphism). Let C be a category with products. Then for all objects A and B , if P and Q are both products of A and B then $P \cong Q$.

Another isomorphic definition of product in **Set** could be the following one:

$$A \times B = \{ (b, a) \mid a \in A \text{ and } b \in B \}$$

or this one

$$A \times B = \{ (b, a, a) \mid a \in A \text{ and } b \in B \}$$

3.5.2 Coproducts

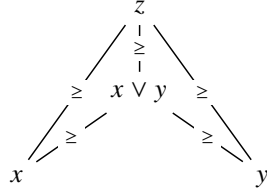
In logic, the *or* operator is introduced by stating that if Γ is a set of true propositions that entails A , or if Γ entails B , then $\Gamma \vdash A \vee B$. This is called the introduction rule of the coproduct:

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} \quad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B}$$

The coproduct has one elimination rule given by:

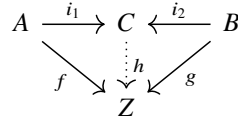
$$\frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash C \quad \Gamma, B \vdash C}{\Gamma \vdash C}$$

In algebra, a set with this structure is called an *upper semilattice*. That is, a set X such that for every $a, b \in X$, there exists an element $a \vee b$, called the *join* or the *least upper bound* of a and b , with the property that for every other upper bound z (i.e., $z \geq a$ and $z \geq b$), we have $z \geq a \vee b$. This can be visualized as follows:



We now generalize the concept of least upper bound to the categorical notion of a coproduct $A + B$.

Definition 3.5 (Coproduct). *For two objects A and B , the coproduct is an object C equipped with two arrows $i_1 : A \rightarrow C$ and $i_2 : B \rightarrow C$ called injections such that for any object Z with arrows $f : A \rightarrow Z$ and $g : B \rightarrow Z$, there exists a unique morphism $h : C \rightarrow Z$ making the following diagram commute:*



We write $A + B$ for the coproduct of two objects A and B and $[f, g]$ for h .

This definition corresponds to the notion of a disjoint union in **Set**:

$$A + B \triangleq \{(a, 1) \mid a \in A\} \cup \{(b, 2) \mid b \in B\},$$

where the injections i_1 and i_2 are defined as:

$$i_1(a) = (a, 1) \quad \text{and} \quad i_2(b) = (b, 2).$$

For any element $f : A \rightarrow Z$ and $g : B \rightarrow Z$, the unique morphism $[f, g]$ is defined by:

$$[f, g](x) = \begin{cases} f(a) & \text{if } x = (a, 1), \\ g(b) & \text{if } x = (b, 2). \end{cases}$$

The existence condition ensures that every element of A and B is included in the coproduct (no omission), while the uniqueness condition ensures that there are no additional elements in $A + B$ (no duplication).

Notice that we could define the coproduct in many ways, but as long as the categorical definition is satisfied, all these definitions are isomorphic. The reader should verify this by proving the following proposition:

Proposition 3.2 (Coproducts are unique up to isomorphism). *Let C be a category with coproducts. Then for all objects A and B , if C and D are both coproducts of A and B , then $C \cong D$.*

3.5.3 Exponentials

In logic, the introduction rule of the implication is typically written as follows:

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \subset B}$$

where $\subset$ is how traditionally implication has been written and is reminiscent of the fact that the information A provides is a subset of the information that B provides. The introduction rule means that if we can derive B from a context Γ and a proof of A then we can derive that given Γ we can prove A implies B . The elimination rule of the implication is called “modus ponens” rule is typically written as follows:

$$\frac{\Gamma \vdash A \subset B \quad \Gamma \vdash A}{\Gamma \vdash B}$$

meaning that if Γ entails A implies B and we also have that Γ entails A then we can derive B .

The algebraic structure modelling this operator is called an *Heyting algebra*. An Heyting algebra consists of a set X with finite meets ($\wedge$) and exponents ($\Rightarrow$) such that the following universal property holds

$$z \wedge x \leq y \iff z \leq x \Rightarrow y$$

for all x, y, z .

This time the categorical definition will not match exactly the abstract definition of the Heyting algebra. However we can show we can reduce the following definition to the one above for preorders.

Definition 3.6 (Exponentials). *An exponential represents the internal function type of a language. The triangular diagram in (1) defines the universality property of an exponential. This is an object which we denote by B^A , for $A, B \in \mathcal{C}$ such that there exists an evaluation map $\epsilon : B^A \times A \rightarrow B$ which intuitively applies the input of type A to a function which takes A to B and such that for every map $Z \times A \xrightarrow{f} B$ there exists a unique map $Z \xrightarrow{\Lambda f} B^A$ such that the diagram in (1) commutes.*

$$\begin{array}{ccc} B^A & & B^A \times A \xrightarrow{\epsilon} B \\ \Lambda f \uparrow & \nearrow f & \\ Z & & Z \times A \end{array} \quad (1)$$

Notice, here Λ can be seen as the currying operation taking maps of type $Z \times A \rightarrow B$ to maps to type $Z \rightarrow B^A$. Conversely, if we happen to have a map $Z \xrightarrow{g} B^A$ then we can easily construct a map $Z \times A \xrightarrow{Vg} B$ by $\epsilon \circ \langle g \circ \pi_1, \pi_2 \rangle$. Thus it can be proven that there is a correspondence of arrows, in other words, a correspondence of homsets

$$\Lambda : C(Z \times A, B) \cong C(Z, B^A) : V$$

which corresponds to our definition of Heyting exponential and it is the categorical definition of currying and uncurrying.

3.6 Exercises

1. In **Set**, a subobject of the NNO is a set $P \subseteq \mathbb{N}$ with two maps $1 \rightarrow P$ and $P \rightarrow P$. Show that there is no proper subobject of the natural numbers in **Set**, in other words, prove that any subobject of $\mathbb{N}$ must be $\mathbb{N}$ itself.
2. Define a program $\text{sum} : \mathbb{N} \rightarrow \mathbb{N}$ using the unique mapping property of the NNO which maps $n \in \mathbb{N}$ into the sum of all numbers from 0 to n
3. Using the properties of the NNO, prove that for all $n \in \mathbb{N}$, the sum of the first n natural numbers is given by the formula:

$$\sum_{k=0}^n k = \frac{n(n+1)}{2}.$$

4. Prove Proposition 3.1. Hint: you have to prove that for two objects A and B if you have two products P and P' for the same two objects you can derive an isomorphism. This is constructed by using the universality property of products.
5. Prove the following morphisms are isomorphisms by showing there for each of these there exists an inverse using the unique mapping property of the product:

$$\text{unit}_A^\times : A \times 1 \rightarrow A \quad (2)$$

$$\text{symm}_{A,B}^\times : A \times B \rightarrow B \times A \quad (3)$$

$$\text{assoc}_{A,B,C}^\times : A \times (B \times C) \rightarrow (A \times B) \times C \quad (4)$$

6. Construct a map

$$\text{copy}_A : A \rightarrow A \times A$$

which makes a copy of the argument using the unique mapping property of the product

7. Given a map $f : X \rightarrow Y$ define the map $X \times A \rightarrow Y \times A$ such that it applies f to the left argument and leaves the right-hand side intact. Call this map $f \times A$. Similarly define $A \times f : A \times X \rightarrow A \times Y$. Finally, for a map $f : X \rightarrow Y$ and $g : X' \rightarrow Y'$ define a map

$$f \times g : X \times X' \rightarrow Y \times Y'$$

such that f is applied to the first component and g is applied to the second component

8. Prove that for all sets $X \in \mathbf{Set}$, the homset $\mathbf{Set}(1, X)$ is in bijective correspondence with X
9. Let X and Y be two initial objects. Prove that $X \cong Y$. Conclude that initial objects are unique up-to isomorphism

10. Let $0 \xrightarrow{!_A} A$ be the unique morphism from the initial object into an object A and $A \xrightarrow{f} B$ be a morphism. Prove that $f \circ !_A$ is equal to $!_B$.
11. Prove Proposition 3.2. Hint: you need to show that for two objects A and B , if C and C' are both coproducts, you can construct an isomorphism using the universal property of coproducts.
12. Let C be a cartesian closed category with an initial object 0 . Show that for any object X in C , $X \times 0$ is initial. Conclude $X \times 0 \cong 0$.
13. In this exercise you must construct the central part of the soundness proof for the interpretation of the simply typed λ -calculus in a cartesian closed category.
Let C be a cartesian closed category. Show that the interpretation is sound with respect to β and η rules for function types, i.e., show that if

$$\begin{aligned} \Gamma, x : A \vdash t &: B \\ \Gamma \vdash u &: A \\ \Gamma \vdash s &: A \rightarrow B \end{aligned}$$

then the following equalities hold

$$\begin{aligned} \llbracket (\lambda x. t) u \rrbracket &= \llbracket t[u/x] \rrbracket \\ \llbracket \lambda x. (sx) \rrbracket &= \llbracket s \rrbracket \end{aligned}$$

Here $t[u/x]$ means *capture-avoiding substitution* of u for x in t . You do not have to worry about what capture avoiding substitution is, rather you should use Lemma 1 and Lemma 2, which you do not have to prove. (They can be proved by induction on t and s respectively.)

4 Semantics of the Simply Typed λ -Calculus

In this section we look at a categorical semantics for the Simply-Typed λ -calculus (STLC) which was introduced by Alonzo Church in 1940. Originally, the calculus only considered function types, but here we add the unit type, the natural numbers object, finite product and coproducts so that we can show off the category theory we introduced so far.

4.1 Syntax

We first define the syntax of STLC by defining the set of types, the set of terms and the typing judgment relation. The set of types Types also inductively as follows

$$\begin{array}{lll} A, B \in \text{Types} & ::= & \text{unit} \quad (\text{unit type}) \\ & | & \text{nat} \quad (\text{natural numbers}) \\ & | & A \times B \quad (\text{products}) \\ & | & A + B \quad (\text{coproducts}) \\ & | & A \rightarrow B \quad (\text{functions}) \end{array}$$

while the set of λ -terms Terms is inductively defined as follows

$t \in \text{Terms} ::=$	x	(terms variables)
	$\text{zero} \mid \text{succ } (t) \mid \text{fold } t_1 t_2 t_3$	(natural numbers)
	$(t_1, t_2) \mid \text{prj}_1(t) \mid \text{prj}_2(t)$	(products)
	$\text{case } t \text{ of } \{\text{inl}(x) \Rightarrow t_1; \text{inr}(y) \Rightarrow t_2\} \mid \text{inl}(t) \mid \text{inr}(t)$	(coproducts)
	$\lambda x.t \mid t_1 t_2$	(functions)

Equational Laws

Typing Judgment The typing judgement relation $\vdash$ is defined inductively as follows

$$\begin{array}{c}
\frac{(x : A) \in \Gamma}{\Gamma \vdash x : A} \quad \frac{}{\Gamma \vdash () : \text{unit}} \quad \frac{}{\Gamma \vdash \text{zero} : \text{nat}} \quad \frac{\Gamma \vdash t : \text{nat}}{\Gamma \vdash \text{succ } (t) : \text{nat}} \\
\frac{\Gamma \vdash t_1 : \text{nat} \quad \Gamma \vdash t_2 : A \quad \Gamma \vdash t_3 : A \rightarrow A}{\Gamma \vdash \text{fold } t_1 t_2 t_3 : A} \\
\frac{\Gamma \vdash t : A \times B}{\Gamma \vdash \text{prj}_1(t) : A} \quad \frac{\Gamma \vdash t : A \times B}{\Gamma \vdash \text{prj}_2(t) : B} \quad \frac{\Gamma \vdash t_1 : A \quad \Gamma \vdash t_2 : B}{\Gamma \vdash \langle t_1, t_2 \rangle : A \times B} \\
\frac{\Gamma \vdash t : A}{\Gamma \vdash \text{inl}(t) : A + B} \quad \frac{\Gamma \vdash t : B}{\Gamma \vdash \text{inr}(t) : A + B} \\
\frac{\Gamma \vdash t : A + B \quad \Gamma, A \vdash t_1 : C \quad \Gamma, B \vdash t_2 : C}{\Gamma \vdash \text{case } t \text{ of } \{\text{inl}(x) \Rightarrow t_1; \text{inr}(y) \Rightarrow t_2\} : C} \\
\frac{\Gamma \vdash t_1 : A \rightarrow B \quad \Gamma \vdash t_2 : A}{\Gamma \vdash t_1 t_2 : B} \quad \frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x.t : A \rightarrow B}
\end{array}$$

4.2 Semantics

The semantics of a programming language is given by the *semantic function* mapping the syntax into the *meaning* of the language. This function is traditionally denoted by $\llbracket \cdot \rrbracket$ and pronounced the *semantic brackets*. We use the semantic brackets for both the types, terms and the context interpretation. First we need to interpret the contexts. Since these are essentially finite lists we need a category with *finite products*. Thus define $\llbracket \Gamma \rrbracket$ by assuming that every context Γ is a finite list of typed variables $x_1 : A_1, \dots, x_n : A_n$:

$$\llbracket x_1 : A_1, \dots, x_n : A_n \rrbracket = \llbracket A_1 \rrbracket \times \dots \times \llbracket A_n \rrbracket$$

Or equivalently, by interpreting the context by induction on the list:

$$\begin{aligned}
\llbracket \cdot \rrbracket &= 1 \\
\llbracket \Gamma, x : A \rrbracket &= \llbracket \Gamma \rrbracket \times \llbracket A \rrbracket
\end{aligned}$$

which is a more intuitive definition for readers familiar with functional programming.

The interpretation of types is then a function $\llbracket \cdot \rrbracket : \text{Types} \rightarrow \text{Obj}(C)$ from syntactic types into object of a category enforcing the intuition that in categorical semantics we

think of types as objects:

$$\begin{aligned}\llbracket \text{unit} \rrbracket &= 1 \\ \llbracket \text{nat} \rrbracket &= \mathbb{N} \\ \llbracket A \times B \rrbracket &= \llbracket A \rrbracket \times \llbracket B \rrbracket \\ \llbracket A \rightarrow B \rrbracket &= \llbracket B \rrbracket^{\llbracket A \rrbracket}\end{aligned}$$

And, finally, the function $\llbracket \cdot \rrbracket : \text{Terms} \rightarrow \text{Arr}(C)$ interprets a term as a morphism between two objects in the category C . However, to be more precise we only interpret well-typed terms, thus it would be more correct to say that a typing derivation $\Gamma \vdash t : A$ is interpreted as an arrow $\llbracket t \rrbracket$ of type $\llbracket \Gamma \rrbracket \rightarrow \llbracket A \rrbracket$, therefore abusing some notation the correct type of the interpretation would be something like:

$$\llbracket \Gamma \vdash t : A \rrbracket : \llbracket \Gamma \rrbracket \xrightarrow{\llbracket t \rrbracket} \llbracket A \rrbracket$$

For example, if $\Gamma, x : A \vdash t : B$ then the interpretation of t has type

$$\llbracket \Gamma, x : A \vdash t : B \rrbracket : \llbracket \Gamma \rrbracket \times \llbracket A \rrbracket \rightarrow \llbracket B \rrbracket$$

We now define this function by induction on the typing judgement:

$$\begin{aligned}\llbracket \Gamma \vdash () : \text{unit} \rrbracket &= ! \\ \llbracket \Gamma \vdash x : A \rrbracket &= \pi_x \\ \llbracket \Gamma \vdash \text{zero} : \text{nat} \rrbracket &= \text{zero} \circ ! \\ \llbracket \Gamma \vdash \text{succ}(t) : \text{nat} \rrbracket &= \text{succ} \circ \llbracket t \rrbracket \\ \llbracket \Gamma \vdash \text{fold } t_1 \ t_2 \ t_3 : A \rrbracket &= \langle \llbracket \Lambda(t_2) \rrbracket, \Lambda(\llbracket t_3 \rrbracket \circ \text{symm}_\times \circ \epsilon \times \text{copy}) \rrbracket \rangle \circ \llbracket t_1 \rrbracket \\ \llbracket \Gamma \vdash \text{prj}_1(t) : A \rrbracket &= \pi_1 \circ \llbracket t \rrbracket \\ \llbracket \Gamma \vdash \text{prj}_2(t) : B \rrbracket &= \pi_2 \circ \llbracket t \rrbracket \\ \llbracket \Gamma \vdash (t_1, t_2) : A \times B \rrbracket &= \langle \llbracket t_1 \rrbracket, \llbracket t_2 \rrbracket \rangle \\ \llbracket \Gamma \vdash \text{inl}(t) : A \rrbracket &= i_1 \circ \llbracket t \rrbracket \\ \llbracket \Gamma \vdash \text{inr}(t) : A \rrbracket &= i_2 \circ \llbracket t \rrbracket \\ \llbracket \Gamma \vdash \text{case } t \text{ of } \{\text{inl}(x) \Rightarrow t_1; \text{inr}(y) \Rightarrow t_2\} : A \rrbracket &= [\Lambda(\llbracket t_1 \rrbracket \circ \text{symm}_\times), \Lambda(\llbracket t_2 \rrbracket \circ \text{symm}_\times)] \circ \llbracket t \rrbracket \\ \llbracket \Gamma \vdash \lambda x. M : A \rightarrow B \rrbracket &= \Lambda \llbracket M \rrbracket \\ \llbracket \Gamma \vdash MN : B \rrbracket &= \epsilon \circ \langle \llbracket M \rrbracket, \llbracket N \rrbracket \rangle\end{aligned}$$

where $! : \llbracket \Gamma \rrbracket \rightarrow 1$ is the unique map into the terminal object and $n : \Gamma \rightarrow \mathbb{N}$ is the map disregarding the context and returning the number denoted syntactically by $\underline{n}$. In **Set**, for example, there would be a constant map $\lambda \gamma. n$ for each n .

4.3 Exercises

1. Suppose $\Gamma, x : A \vdash t : B$ and $\Gamma \vdash u : A$ are valid typing judgments. Then so is $\Gamma \vdash t[u/x] : B$ and the interpretation of $\Gamma \vdash t[u/x] : B$ is the composite

$$\llbracket \Gamma \rrbracket \xrightarrow{\langle \text{id}, \llbracket u \rrbracket \rangle} \llbracket \Gamma \rrbracket \times \llbracket A \rrbracket \xrightarrow{\llbracket t \rrbracket} \llbracket B \rrbracket$$

2. If $\Gamma \vdash s : C$ is a valid typing judgment, so is $\Gamma, x : A \vdash s : C$ and the following diagram is commutative

$$\begin{array}{ccc}
 \llbracket \Gamma \rrbracket \times \llbracket A \rrbracket & \xrightarrow{\llbracket \Gamma, x : A \vdash s : C \rrbracket} & \llbracket C \rrbracket \\
 \pi_1 \downarrow & \nearrow \llbracket \Gamma \vdash s : C \rrbracket & \\
 \llbracket \Gamma \rrbracket & &
 \end{array}$$

5 Functors and Natural Transformations

In this section we are going to look at functors which are mappings between categories and natural transformations which are mapping between functors.

5.1 Functors

Imagine the you have some object A representing the type for some data and a data type which wraps the elements of A around some structure. For example, this structure may be the lists over A , written $\text{List } A$. Graphically, we may draw a list of A s as a box $\boxed{A}$ with inside some piece of data of type A . Now given a map $f : A \rightarrow B$ we can recursively go through the list and change the data inside using f obtaining a map, which we call $\text{List } f$, of type

$$\boxed{A} \rightarrow \boxed{B}$$

transforming the data inside a list while preserving the structure of the list. Generalising, we can say the box is a *functor* F and the map $F(f)$ is the *functorial action* of F .

Another way of looking at functors is as order-preserving maps between categories. We covered briefly the concept of preorders as particular sets with an additional structure. In a preorder $(X, \leq)$, elements have a sense of order: if $a \leq b$, then a comes before b . A *structure-preserving* map between preorders is called a *monotone function*. This map respects the order of the set X in the following way: if $a \leq b$ in one preorder, then their images still satisfy $f(a) \leq f(b)$ in the other.

Categories are a richer version of a preorder, where instead of just “ $\leq$ ”, we have morphisms (arrows) between objects.

A *functor* is a map between categories that, just like a monotone function, preserves structure. It has two components:

- It takes each object X in one category and assigns it to an object $F(X)$ in another.
- It takes each morphism $f : X \rightarrow Y$ and assigns it to a morphism $F(f) : F(X) \rightarrow F(Y)$, keeping the relationships intact.

with the additional conditions that that it also also respects how morphisms behave, that is it respects identity morphisms and it respects composition. The formal definition follows:

Definition 5.1 (Functor). *Let $\mathcal{C}$ and $\mathcal{D}$ be two categories. A functor $F : \mathcal{C} \rightarrow \mathcal{D}$ is a mapping between categories that associates objects in $\mathcal{C}$ to an object $FC \in \text{Obj}(\mathcal{D})$ and that has additionally a functorial action associating arrows $f \in \text{hom}_{\mathcal{C}}(A, B)$ to arrows $F(f) \in \text{hom}_{\mathcal{D}}(FA, FB)$ which additionally preserves identity and composition:*

$$F(id_A) = id_{FA} \quad F(g \circ f) = F(g) \circ F(f)$$

We are going to look at some examples in great depth whilst we are going to leave others to the reader to look at.

A very popular example of a functor is the “maybe” functor

$$FX = X + 1$$

This can be viewed as the type of computations that either return an element of X , written (**Just** x) or fail, written **Nothing**. In the former case is equivalent to write **inl**(x), in the latter case this is equivalent to writing **inr**($*$). The functorial action is defined as $F(f) = f + 1$ which is a map of type $X + 1 \rightarrow Y + 1$ by applying f to the left part of the coproduct. This is formally defined using the unique mapping property of the coproduct (left as exercise).

Another very popular functor is list functor defined as the least solution to the following domain equation

$$\text{List } A \cong 1 + A \times \text{List } A$$

where $\phi : 1 + A \times \text{List } A \rightarrow \text{List } A$ is an isomorphism. This is recursively defined using a technique we are going to see later in this notes, for now we are just going to assume this is a well-defined inductive definition, i.e. that this isomorphism exists. The empty list $[]$ is represented by the element $\phi \circ \iota_1$, the cons operation $(: :)$ is defined by $\phi \circ \iota_2$.

The functorial action is defined by $\text{List } f = 1 + f \times \text{List } A$, using the functorial action of the coproduct and the product.

Finally, the stream functor is defined as the greatest solution to the following equation

$$\text{Str } A \cong A \times \text{Str } A \tag{5}$$

where ϕ is the operation $\text{cons} : A \times \text{Str } A \rightarrow \text{Str } A$, the head $\text{head} : \text{Str } A \rightarrow A$ is defined as $\pi_1 \circ \phi^{-1}$ and the tail $\text{tail} : \text{Str } A \rightarrow \text{Str } A$ is defined as $\pi_2 \circ \phi^{-1}$.

The functorial action is defined by $\text{Str } (f) = f \times \text{Str } A$ using the functorial action of the coproduct and the product.

We list some more examples of functors, the interested reader is invited to work out the details of these definitions.

Example 5.1. *Some popular functors include:*

- *the identity functor $FX = X$ with the obvious functional action*
- *the product functor (on one variable) $FX = A \times X$ with functorial action $F(f) = A \times f$*
- *the exponential $FX = X^A$ with the functorial action given by post-composition, that is $F(f) = \Lambda(f \circ \epsilon)$ where $\epsilon : X^A \times A \rightarrow X$ is the evaluation map*

- the homset functor $FX = C(A, X)$. This will be discussed more in depth later

From the definition of functor it follows almost directly that every functor preserves isomorphisms:

$$A \cong B \Rightarrow FA \cong FB$$

the converse is not true unless the functor is surjective on morphisms, that is the functor is *full*. When the functorial action is injective the functor is called *faithful*.

Proposition 5.1. *A fully faithful functor F preserves and reflects isomorphisms:*

$$A \cong B \iff FA \cong FB$$

Exercise 5.1. *Prove Proposition 5.1*

5.2 Natural Transformations

Suppose that you want to write an algorithm which converts a computation on X which may fail into the list of values X this computation can produce. This means we have to write a function from **Maybe** X to **List** X regardless of what X . The most natural way to do this is by mapping **Nothing** to $[],$ **Just** x to a finite list of x s, thus one possible choice is $[x].$

This transformation is agnostic to the type of values. Since it holds for all type of values the conversion is not arbitrary but follows a deep categorical pattern. Thus, turning **Maybe** into **List** this way is a natural transformation.

Definition 5.2 (Natural Transformation). *For two functors $F, G : C \rightarrow \mathcal{D}$, a natural transformation is a family of arrows $\phi_A : FA \rightarrow GA$ such that for every arrow $f : A \rightarrow B$ the following diagram commutes:*

$$\begin{array}{ccc} FA & \xrightarrow{\phi_A} & GA \\ F(f) \downarrow & & \downarrow G(f) \\ FB & \xrightarrow{\phi_B} & GB \end{array}$$

The *category of functors*, denoted as $[C, \mathcal{D}]$, is a category where the objects are functors $F : C \rightarrow \mathcal{D}$, where C and $\mathcal{D}$ are two categories. The morphisms in $[C, \mathcal{D}]$ are natural transformations between functors.

An isomorphism in $[C, \mathcal{D}]$ is a *natural isomorphism*, which means that for a natural transformation $\phi : F \rightarrow G$, each $\phi_X : F(X) \rightarrow G(X)$ is an isomorphism in $\mathcal{D}$, i.e., there exists an inverse $\phi_X^{-1} : G(X) \rightarrow F(X)$ such that $\phi_X \circ \phi_X^{-1} = \text{id}_{G(X)}$ and $\phi_X^{-1} \circ \phi_X = \text{id}_{F(X)}$. This implies that F and G are isomorphic functors in $[C, \mathcal{D}]$.

5.3 The Homset Functor

Given a (locally small) category C^1 , the hom-set $C(A, B)$, for any objects A, B , induces two functors.

¹A category is **small** if its collection of objects forms a set, and it is **locally small** if, for any two objects A, B , the collection of arrows between them forms a set. However, we do not delve into size issues in these notes.

The first is the *covariant hom-functor* $C(A, -) : C \rightarrow \mathbf{Set}$, which assigns to each object B the set of morphisms $C(A, B)$. Given an arrow $f : B \rightarrow B'$ in C , the functor maps it to the function $C(A, f) : C(A, B) \rightarrow C(A, B')$, which acts by *post-composition*: each arrow $g : A \rightarrow B$ is sent to $f \circ g : A \rightarrow B'$.

The second is the *contravariant hom-functor* $C(-, B) : C^{\text{op}} \rightarrow \mathbf{Set}$, which assigns to each object A the set $C(A, B)$. This functor is *contravariant* because it reverses arrows: a morphism $f : A \rightarrow A'$ in C is mapped to the function $C(f, B) : C(A', B) \rightarrow C(A, B)$, which acts by *pre-composition*: each arrow $g : A' \rightarrow B$ is sent to $g \circ f : A \rightarrow B$.

Functors which are *naturally isomorphic* to the functor $C(A, -)$ for some $A \in \text{Obj}(C)$ are called *representable functors*.

For example, the stream functor $\text{Str} : \mathbf{Set} \rightarrow \mathbf{Set}$ is isomorphic to the functor $\text{hom}_{\mathbf{Set}}(\mathbb{N}, -)$, i.e. the following isomorphism is natural in A

$$\text{Str } A \cong \mathbf{Set}(\mathbb{N}, A) \quad (6)$$

Exercise 5.2. Prove the natural isomorphism (6) by defining a natural transformation $\phi_A : \text{Str } A \rightarrow \mathbf{Set}(\mathbb{N}, A)$ and its inverse.

Exercise 5.3. Show that the type of lists as defined below

$$\text{List } A \cong 1 + A \times \text{List } A \quad (7)$$

is not representable.

Exercise 5.4. Let C be a locally small category, and let A, B, P be object of C . Show that P is a product of A, B , i.e. there is a product diagram

$$A \longleftarrow P \longrightarrow B$$

if and only if there is an isomorphism

$$C(X, P) \cong C(X, A) \times C(X, B)$$

that is natural in X , in other words, iff the functors

$$\begin{aligned} C(-, P) : C^{\text{op}} &\rightarrow \mathbf{Set} \\ C(-, A) \times C(-, B) : C^{\text{op}} &\rightarrow \mathbf{Set} \end{aligned}$$

are (naturally) isomorphic.

Exercise 5.5. Let $\mathcal{D}$ be a locally small category, and let A, B, E be objects in $\mathcal{D}$, then there is an isomorphism

$$\mathcal{D}(X, E) \cong \mathcal{D}(X \times A, B)$$

natural in X if and only if E is an exponential object of B to the A (i.e. there exists an $\epsilon : E \times A \rightarrow B$ such that (E, ϵ) is an exponential).

When using the 'only if' direction of the lemma, you should apply standard practice: construct the isomorphism, and then say the magic words: "it is obvious that this is natural in X ".

Exercise 5.6. Let C be a locally small category with finite products, let $F : C \rightarrow C$ and $G : \mathcal{D} \rightarrow C$ be a part of an equivalence of categories, and let A, B be the objects of $\mathcal{D}$. Then $F(1)$ is a terminal object in $\mathcal{D}$ and

$$A \times B \stackrel{\Delta}{=} F(GA \times GB)$$

defines a product of A and B .

5.4 The Yoneda Lemma

Consider a typed language with a unary constructor R that follows the typing rule

$$\frac{\Gamma \vdash t : A}{\Gamma \vdash R(t) : B}. \quad (8)$$

Our goal is to provide a semantic interpretation $\llbracket \cdot \rrbracket$ for this language by induction on the typing judgment $\Gamma \vdash t : A$, such that terms are mapped to morphisms $\llbracket \Gamma \rrbracket \xrightarrow{\llbracket t \rrbracket} \llbracket A \rrbracket$, assuming $\llbracket \cdot \rrbracket$ is also separately defined for contexts and types.

To interpret (8), we apply induction on the typing judgment. By assumption, we already have a morphism $\llbracket \Gamma \rrbracket \xrightarrow{\llbracket t \rrbracket} \llbracket A \rrbracket$, and our task is to construct a morphism $\llbracket \Gamma \rrbracket \xrightarrow{\llbracket R(t) \rrbracket} \llbracket B \rrbracket$.

For simplicity, in the remainder of this section, we omit semantic brackets where clear from context. For example, we assume A represents $\llbracket A \rrbracket$, and similarly, $t : \Gamma \rightarrow A$ represents the interpretation of t .

Determining the correct morphism in this situation can be nontrivial due to the need to manage context correctly. However, a particular instantiation of the Yoneda lemma provides a canonical way to construct the desired morphism. Specifically, given a morphism $t : \Gamma \rightarrow A$ and a morphism $R : A \rightarrow B$, the Yoneda lemma ensures the existence of a canonical morphism $\Gamma \rightarrow B$.

$$C(A, B) \cong \text{Nat}(C(-, A), C(-, B))$$

Let $R : A \rightarrow B$ be the interpretation of R . By the isomorphism, the corresponding natural transformation satisfies $\phi(R, t) = F(t)(R) = C(t, B)(R)$, which simply reduces to $R \circ t$. Thus, the interpretation of $R(t)$ is given by composition: $R \circ t$.

The Yoneda Lemma is a fundamental result in category theory that describes how functors of type $C^{\text{op}} \rightarrow \mathbf{Set}$ interact with natural transformations. These kind of functors are called *presheaves*. Given a locally small category C and a functor $F : C^{\text{op}} \rightarrow \mathbf{Set}$, the Yoneda Lemma states that for any object A in C , there is a natural isomorphism

$$F(A) \cong \text{Nat}(C(-, A), F)$$

where $\text{Nat}(C(-, A), F)$ denotes the set of natural transformations from the hom-functor $C(-, A)$ to F , and $F(A)$ is the value of F at A . This result tells us that giving a natural transformation $\eta : C(-, A) \rightarrow F$ is equivalent to specifying an element of $F(A)$. In other words, the presheaf F is completely determined by how it interacts with the representable functors $C(-, A)$.

To see why this holds, we construct a correspondence between elements of $F(A)$ and natural transformations $C(-, A) \rightarrow F$. Given a natural transformation $\phi : C(-, A) \rightarrow F$, we obtain an element of $F(A)$ by evaluating ϕ_A at the identity morphism, that is, setting $x = \phi_A(\text{id}_A) \in F(A)$. Conversely, given an element $x \in F(A)$, we define a natural transformation ϕ by setting $\phi_B(f) = F(f)(x)$ for any morphism $f : B \rightarrow A$. The fact that these two operations are inverse to each other establishes the natural isomorphism.

5.5 The Category of Categories

The *category of categories*, denoted **Cat**, is the category whose objects are *small categories* and whose morphisms are functors between them. A category consists of objects, morphisms (arrows), composition of morphisms that is associative, and identity morphisms. Since the collection of *all* categories is too large to form a category, **Cat** typically consists of *small categories*, meaning those whose collections of objects and morphisms form sets rather than proper classes. The morphisms in **Cat** are *functors*, which map objects and morphisms between categories while preserving composition and identity morphisms. Functors compose naturally: if $F : C \rightarrow D$ and $G : D \rightarrow E$ are functors, their composition $G \circ F : C \rightarrow E$ is also a functor. The identity morphisms in **Cat** are identity functors, which map each object and morphism to itself. In addition to functors, there exist *natural transformations*, which provide structure-preserving ways to transition between functors. This makes **Cat** more than just a category – it is actually a *2-category*, where objects are categories, 1-morphisms are functors, and 2-morphisms are natural transformations between functors. This additional structure allows for deeper relationships between categories, making **Cat** fundamental in higher category theory.

Having the categories of categories allows us to talk about isomorphisms of categories. Following the definition of isomorphism these are the categories such that there exists a functor $F : C \rightarrow D$ along with an inverse functor f^{-1} such that F^{-1} is F 's inverse.

Example 5.2 (**Set** is not isomorphic to **Set**^{op}). Assume we have an isomorphism (of categories) $F : \mathbf{Set} \rightarrow \mathbf{Set}^{\text{op}}$ and consider a map $f : A \rightarrow \emptyset$ in **Set**.

Since $\emptyset$ is initial in **Set** then $F\emptyset$ should be terminal in **Set**^{op}. Now $Ff : FA \rightarrow F\emptyset$ is a map in **Set**^{op} and because $F\emptyset$ is terminal we have many more arrows $FA \rightarrow F\emptyset$ than $A \rightarrow \emptyset$. The reader is invited to work out the details as exercise.

Example 5.3 (**Rel** is isomorphic to **Rel**^{op}). The category **Rel** of relations has as objects sets X, Y, Z and as arrows $f : X \rightarrow Y$ relations $f \subseteq X \times Y$. Define now two functors $F : \mathbf{Rel} \rightarrow \mathbf{Rel}^{\text{op}}$ and $G : \mathbf{Rel}^{\text{op}} \rightarrow \mathbf{Rel}$ such that they are one the inverse of the other. The isomorphism is given by the functor $FX = X$ with functorial action $F(R : X \rightarrow Y)(y \in Y) = \{x \mid x R y\}$ along with its inverse $GX = X$ with functorial action $G(R : Y \rightarrow X)(y \in Y) = \{x \mid y R x\}$

Verify that this is an isomorphism. (You first need to define the composition of two morphisms (relations) in **Rel**)

5.6 Equivalence of Categories

5.7 Exercises

1. Consider the category $\mathbf{Set}^{\text{op}}$, the opposite category of $\mathbf{Set}$, and the category of *complete atomic Boolean algebras* (**CABA**).
 - (a) Define the objects and morphisms in $\mathbf{Set}^{\text{op}}$ and **CABA**.
 - (b) Show that there exists a *contravariant functor* $F : \mathbf{Set}^{\text{op}} \rightarrow \mathbf{CABA}$ that:
 - Takes a set A to the Boolean algebra $\mathcal{P}(A)$, the power set of A .
 - Takes a function $f : A \rightarrow B$ in $\mathbf{Set}^{\text{op}}$ and maps it to a *reversed* Boolean algebra homomorphism $h : \mathcal{P}(B) \rightarrow \mathcal{P}(A)$.
 - (c) Prove that this functor establishes an *equivalence of categories* between $\mathbf{Set}^{\text{op}}$ and **CABA**.

6 Limits and Colimits

We are going to proceed up the ladder of abstraction and generalise the notion of arbitrary products and coproducts.

Given a set I and a family of sets indexed by I , $\{A_i\}$ we can form the *dependent product* of these sets $\prod_{i \in I} A_i$ which informally is the product of all sets A_i

$$A_1 \times A_2 \times A_3 \times \cdots \times A_n \times \dots$$

Dually we can form the *dependent sum* is denoted by $\sum_{i \in I} A_i$ and can be written informally as follows:

$$A_1 + A_2 + A_3 + \cdots + A_n + \dots$$

Limits and Colimits are just generalisation of these concepts.

Definition 6.1 (Cone). *For a diagram $D : I \rightarrow \mathcal{D}$, a cone is an object C such that there exists a family of maps $\pi_i : C \rightarrow D_i$ and such that*

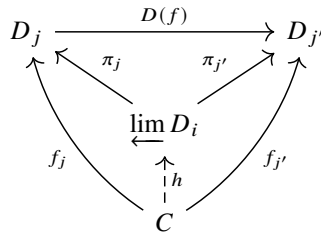
$$D(f) \circ \pi_i = \pi_j \quad (9)$$

for every $f : i \rightarrow j$.

Definition 6.2 (Limit). *For a diagram $D : I \rightarrow \mathcal{C}$, a limit is a universal cone P , that is a cone that for any other cone for D , say C , there exists a unique arrow $h : C \rightarrow \lim_{i \in I} D_i$ such that*

$$\pi_i \circ h = f_i \quad (10)$$

for every $i \in I$. The picture below summarises the situation:



We denote the limit of a diagram D by $\lim_{\leftarrow i \in \mathcal{I}} D_i$

Exercise 6.1. The notion of colimit is obtained by reversing the arrows. Derive it by exercise.

Dependent Products and Sums The dependent product and dependent sum are just limits and colimits respectively where the category $\mathcal{I}$ is discrete¹. In this particular case the coherence conditions (9) on the projections are vacuous because there are no meaningful arrows in the index category. This means, for example in the case of the product, that every tuple is in the limit as opposed to just those that satisfy the coherence condition.

Powers and Copowers Assume that our diagram $D : \mathcal{I} \rightarrow \mathcal{C}$ is constant additionally to the category $\mathcal{I}$ being discrete and assume $D(i) = A$ for some $A \in \mathcal{C}$. Then we have another special case where the dependent product becomes a product of the same object $\prod_{i \in \mathcal{I}} A$ which is called the *power* and denoted by $A^{\mathcal{I}}$. Note that this is in general different from the exponential because $\mathcal{I}$ is a category, not an object. However, in **Set** the power is isomorphic to the function space $\mathcal{I} \rightarrow A$ since $\mathcal{I}$ can be viewed as a set because it is discrete¹.

Similarly, the *copower* is a special case of dependent sum when D is constant and is denoted by $\mathcal{I} \bullet A$. Moreover in **Set**, this is just the pair $\mathcal{I} \times A$ with $\mathcal{I}$ again viewed as a set.

7 Algebraic Data Types

Streams as Limits Assume $\mathcal{C}$ is a category with all limits. The streams over A from definition (5) are obtained as the limit for a diagram. We first define the functor $F : \mathbf{Set} \rightarrow \mathbf{Set}$ by $FX = A \times X$ giving the non-recursive shape of the streams. Then we define the ω^{op} -chain of approximations represented by the diagram $D : \omega^{\text{op}} \rightarrow \mathcal{C}$ defined on object by $D(1) = 1$ and $D(n+1) = F^n 1$ and on arrows by $D(1 \leq 2) = !$ (the unique map to the terminal object) and $D(n \leq n+1) = F^n !$. The limit of this ω^{op} -chain is the type of coinductive streams as depicted below:

$$\begin{array}{c}
 \lim F^n 1 \\
 \swarrow \pi_1 \quad \searrow \pi_2 \quad \swarrow \pi_3 \quad \searrow \pi_{n+1} \\
 1 \xleftarrow{!} F1 \xleftarrow{F!} F^2 1 \xleftarrow{F^2!} \dots \xleftarrow{F^{n-1}!} F^n 1 \xleftarrow{F^n!} \dots
 \end{array}$$

Lists as Colimits Assume $\mathcal{C}$ is a category with all colimits. The lists over A from definition (7) are obtained as the colimit for a diagram. We first define the functor $F : \mathbf{Set} \rightarrow \mathbf{Set}$ by $FX = 1 + A \times X$ giving the non-recursive shape of the lists. Then we

¹The category of objects and identity arrows

¹Notice that $\mathcal{I}$ needs to be small for the collection of objects to be a set, but again, size issues are not really an issue for the focus of these notes

define the ω -chain of approximations represented by the diagram $D : \omega \rightarrow C$ defined on object by $D(1) = 0$ and $D(n+1) = F^n 0$ and on arrows by $D(1 \leq 2) = !$ (the unique map from the initial object) and $D(n \leq n+1) = F^n !$. The colimit of this ω -chain is the type of coinductive streams as depicted below:

$$\begin{array}{ccccccc}
 & & & & \lim F^n 0 & & \\
 & & \nearrow & \nearrow & \nearrow & \nwarrow & \\
 0 & \xrightarrow{!} & F1 & \xrightarrow{F!} & F^2 1 & \xrightarrow{F^2!} & \cdots \xrightarrow{F^{n-1}!} F^n 1 \xrightarrow{F^n!} \cdots \\
 & \nearrow & \nearrow & \nearrow & \nearrow & \nwarrow & \\
 & \text{inj}_1 & \text{inj}_2 & \text{inj}_3 & & \text{inj}_{n+1} &
 \end{array}$$

8 Adjunctions

Almost every universality property comes from an adjunction¹ and certainly all the constructions seen so far are in fact an instance of an adjunction.

8.0.1 Adjunctions

Given two functors $L : \mathcal{D} \rightarrow C$ and $R : C \rightarrow \mathcal{D}$ and *adjunction* is an isomorphism of homsets

$$[\cdot] : C(LA, B) \cong \mathcal{D}(A, RB) : [\cdot]$$

which is furthermore natural in A and B . Here $[\cdot]$ and $[\cdot]$ are the maps witnessing the isomorphism. The adjunction is usually depicted as follows

$$\begin{array}{ccc}
 C & \xleftarrow{L} & \mathcal{D} \\
 & \text{⌞} & \\
 & \xrightarrow{R} &
 \end{array}$$

We say that that L is *left adjoint* to R , and viceversa, R right adjoint to L . However, you draw the adjunction the turnstile points towards the left-adjoint as indicated by $L \vdash R$.

As a consequence of the isomorphism we have that for all $f : LA \rightarrow B$ and $g : A \rightarrow RB$ there is a correspondence of arrows:

$$[f] = g \iff f = [g]$$

moreover, the natural isomorphism gives rise to the *fusion laws*. For $a : A' \rightarrow A$, $b : B \rightarrow B'$, $f : LA \rightarrow B$ and $g : A \rightarrow RB$

$$R(b) \cdot [f] = [b \cdot f] \tag{11}$$

$$[f] \cdot a = [f \cdot L(a)] \tag{12}$$

$$b \cdot [g] = [R(b) \cdot g] \tag{13}$$

$$[g] \cdot L(a) = [g \cdot a] \tag{14}$$

This is really all about adjunctions. All the other definitions and constructions are equivalent to this one. Furthermore, this material is very well covered elsewhere (e.g. in Awodey's book [1]) so we will not be covering it further.

¹More in general a universal map is the initial object in the comma category $X \downarrow F$, but that is not our concern here.

8.1 Instances of Adjunctions

8.1.1 Initial and Terminal Objects

Assume that we have a category $\mathbf{1}$ with only one object $*$ and one arrow, the identity arrow id_* . Then the universality property of the initial and terminal object can be then rephrased in terms of adjunctions

$$C \begin{array}{c} \xleftarrow{0} \\ \xrightarrow[\Delta]{+} \end{array} \mathbf{1} \begin{array}{c} \xleftarrow{\Delta} \\ \xrightarrow[1]{+} \end{array} C$$

where 0 is the constant functor returning the initial object and 1 is the functor returning the terminal object while Δ is the constant functor returning the element $*$.

We can prove that if the above are indeed adjunctions then the functors 0 and 1 must given the initial and terminal object respectively. The isomorphism given by the adjunction with 0 as left adjoint is as follows:

$$[\cdot] : \text{hom}_C(0, Y) \cong \text{hom}_{\mathbf{1}}(*, *) : [\cdot]$$

while for the terminal object the adjunction is given by the following natural isomorphism:

$$[\cdot] : \text{hom}_{\mathbf{1}}(*, *) \cong \text{hom}_C(X, 1) : [\cdot]$$

Exercise 8.1. *Compute the two naturality conditions and derive the fusion laws.*

We now proceed with the universal property of products and coproducts.

8.1.2 (Co)products and Products

Similary the coproduct arises as the left adjoint of the diagonal functor $\Delta : C \rightarrow C \times C$ which is defined as $\Delta X = (X, X)$ while the product arises as the right adjoint of the functor Δ :

$$C \begin{array}{c} \xleftarrow{+} \\ \xrightarrow[\Delta]{+} \end{array} C \times C \begin{array}{c} \xleftarrow{\Delta} \\ \xrightarrow[\times]{+} \end{array} C$$

Exercise 8.2. *Derive the fusion laws and conclude these are exactly those of the product and coproduct respectively.*

8.1.3 Exponentials

The exponential is given by the right adjoint of the functor $(- \times A)$. That is the functor $(-)^A$:

$$C \begin{array}{c} \xleftarrow{(-) \times A} \\ \xrightarrow[(-)^A]{+} \end{array} C$$

The isomorphism induced by this adjunction is stated as follows:

$$[\cdot] : \text{hom}_C(X \times A, Y) \cong \text{hom}_C(X, Y^A) : [\cdot]$$

Notice that here $[\cdot]$ and $[\cdot]$ are respectively the curry and uncurry operations in functional programming.

8.1.4 (Co)Limits

Limits and colimits stem from adjunctions too.

The limit, in particular, extends to a functor from the category of diagrams C^I to the category C simply taking a diagram and returning its limit. This functor is right adjoint to the diagonal functor $\Delta : C \rightarrow C^I$ defined as $\Delta X I = X$ mapping an object to the constant diagram which returns that object. Dually, colimits are left-adjoints to the diagonal functor. The situation is depicted below:

$$C \begin{array}{c} \xrightarrow{\lim} \\ \perp \\ \xrightarrow{\Delta} \end{array} C^I \begin{array}{c} \xleftarrow{\Delta} \\ \perp \\ \xleftarrow{\lim} \end{array} C$$

At this point the reader should notice that the similarity between initial and terminal, coproducts and products and colimits and limits. Indeed initial objects and coproducts are special cases of colimits while terminal objects and products are special cases of limits.

9 Semantics of Predicative Polymorphism

Polymorphism is a core feature of most programming languages allowing developers to craft generic programs which are agnostic to the specifics of the types.

In programming languages like C, we can define algorithms that work for lists that contain different types, but we have to define the algorithm for each and everyone of the specific type we want to handle. This type of polymorphism is called *ad-hoc polymorphism*. *Predicative polymorphism* allows the programmer to write one function that is *parametric* on the type or, in words, it takes a generic type with the promise that the data of that type will not be inspected. Therefore, this kind of algorithm can be said to be agnostic as to the type of data structure given. This allows for code reusability since a polymorphic program can be written only once and can work at all types.

This flexibility provides a layer of confidence regarding the properties of the code. Philip Wadler's paper [10] offers a comprehensive exploration of these properties.

As an example, in a total language there is no program of type $\forall \alpha. \alpha$ since such a program would need to yield a value of an arbitrary type α . Consequently, $\forall \alpha. \alpha$ can be viewed as the empty type 0, which, in a total language¹, acts as the initial object.

Similarly, $\forall \alpha. \alpha \rightarrow \alpha$ has only one inhabitant, namely the identity function $\Lambda \alpha. \lambda x. x$, with Λ abstracting a type variable and λ abstracting a term variable.

Now the **reverse** function, reversing a list, is indeed a polymorphic function since it needs not inspecting the content of the single element in a list:

$$\forall \alpha. \text{List } \alpha \rightarrow \text{List } \alpha \quad (15)$$

This type grants universality by reversing elements without inspecting their specifics. Conversely, any sorting algorithm on lists needs to know that the inner type inside the

¹There are of course issues with non-termination but we will not address them here

list as some kind of ordering associated with it, rendering (15) unsuitable for quicksort or mergesort, for instance.

Once we defined a polymorphic function, we have the liberty to instantiate it with any desired type, even itself. This style of polymorphism is called *impredicative* and was first introduced by Girard in 1972 [2] in the context of proof theory and then proposed by Reynolds in 1983 [8] as a programming language known was *System F* or *second-order λ -calculus*.

For instance, the `reverse` function might operate on elements of type $\mathbb{N}$ or η , but also on the type (15) itself.

While impredicative polymorphism is convenient in programming languages, it poses a significant foundational problem when seeking for a semantic model as we did in Section 4. As we have seen the task of seeking a denotational model is to find a universe of sets $\mathcal{U}$ and interpreting types with n free variables as maps $\mathcal{U}^n \rightarrow \mathcal{U}$. Assuming no free variables, we would like to interpret $\forall\alpha.\alpha$ as a set. Naively we could try to interpret it as the product of sets indexed over sets. Now, the denotation of $\forall\alpha.\alpha$ would be a set for the interpretation to work and, as a result, we would have that the product of all sets would be the *set containing all sets* which is a paradoxical statement known as the Russell's paradox, suggesting that such a set cannot exist. This fact was discovered by Reynolds in 1984 [9].

Models of impredicative polymorphism have eventually been found by Pitts [7] who showed this in constructive set theory.

In these notes, we are going to avoid this problem and restrict ourselves to *predicative polymorphism* which is a variant of impredicative polymorphism where polymorphic types (*polytypes* or *type schemes*) can only be instantiated with non-polymorphic types (*monotypes*).

9.0.1 Syntax

The language we are going to define is called ML_0 . We first define the syntax by defining the set of monotypes, the set of polytypes, the set of terms and the typing judgment relation. The set of types monotypes and polytypes is defined inductively as follows:

$$\begin{array}{lll}
 A, B \in \text{Types}_0 & ::= & \alpha \quad (\text{type variables}) \\
 & | & \text{unit} \quad (\text{unit type}) \\
 & | & \text{nat} \quad (\text{natural numbers}) \\
 & | & A \times B \quad (\text{products}) \\
 & | & A \rightarrow B \quad (\text{functions}) \\
 \tau \in \text{Types}_1 & ::= & A \mid \forall\alpha.\tau \quad (\text{predicative polymorphism})
 \end{array}$$

Notice that polytypes are of the form $\forall\alpha.\forall\beta.A$ where A is a monotype. Thus types of the form $\forall\alpha.(\forall\beta.\beta) \rightarrow \alpha$ for example are not allowed.

The set of λ -terms `Terms` is inductively defined as follows:

$t \in \text{Terms}$	$::=$	x	(terms variables)
		$()$	(unit)
		$\underline{n}$	(natural numbers)
		$(t_1, t_2) \mid \text{prj}_1(t) \mid \text{prj}_2(t)$	(products)
		$\lambda x. t \mid t_1 t_2$	(functions)
		$\text{let } x = t_1 \text{ in } t_2 \mid \Lambda \alpha. t \mid t@A$	(polymorphism)

where Λ is a binder which abstracts over type variables, the **let** binds a program M of a polytype inside a program N which returns a monotype. This allows the programmer to write programs such as

$\text{let } x = id \text{ in } x@unit : unit \rightarrow unit$

where $id : \forall \alpha. \alpha \rightarrow \alpha$ and the constructor $M@A$ is the application for terms that have a polytype as a type.

Note that since we have defined types using to separate layers for grammars we can constrain the types that we are going to apply to polymorphic programs. In fact, in the constructor $M@A$ only monotypes are allowed.

We first define a judgment for the contexts and the types. Technically, there are two judgments for types, one for the monotypes and one for the polytypes, but we adopt the same notation for both of them as it should be clear from the context which judgment we mean. First a type context is a list of variables. The $unit$ type can be typed in any well-formed context and a type variable can be typed in any well-formed context that contains it. Finally the polymorphic types $\forall \alpha. \tau$ are well-typed if the body τ is open in α and well-typed: The context for term variables is instead a list of pairs $x : \tau$ implying that variables are of poly-typed and therefore can also be mono-typed.

$$\Gamma = (x_1 : \tau_1, \dots, x_n : \tau_n)$$

We now define the typing judgment for terms. Again, there are technically two typing judgments, the first for mono-typed programs and the second for poly-typed programs. Once again, it should be clear from the context which judgment we are using:

$$\begin{array}{c}
\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau} \quad \frac{}{\Gamma \vdash () : unit} \quad \frac{}{\Gamma \vdash \underline{n} : nat} \\
\frac{\Gamma \vdash t : A \times B}{\Gamma \vdash \text{prj}_1(t) : A} \quad \frac{\Gamma \vdash t : A \times B}{\Gamma \vdash \text{prj}_2(t) : B} \quad \frac{\Gamma \vdash t_1 : A \quad \Theta \mid \Gamma \vdash t_2 : B}{\Gamma \vdash \langle t_1, t_2 \rangle : A \times B} \\
\frac{\Gamma \vdash t_1 : A \rightarrow B \quad \Gamma \vdash t_2 : A}{\Gamma \vdash t_1 t_2 : B} \quad \frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x. t : A \rightarrow B} \\
\frac{\Gamma \vdash t_1 : \tau \quad \Gamma, x : \tau \vdash t_2 : A}{\Gamma \vdash \text{let } x = t_1 \text{ in } t_2 : A} \\
\frac{\Gamma \vdash t : \tau}{\Gamma \vdash \Lambda \alpha. t : \Pi \alpha. \tau} \text{ if } \alpha \notin \text{Ftv}(\Gamma) \quad \frac{\Gamma \vdash t : \Pi \alpha. \tau}{\Gamma \vdash t@A : \tau[A/\alpha]}
\end{array}$$

The first typing rule is for variables. Notice once again that variables can be typed with a poly-type and therefore the context should accommodate that. Additionally the polytype itself has to be well-typed in the context Θ .

We now justify some choices behind the type system.

First notice the variable rule which says that variables' types can be open. The first example that justifies this is the polymorphic identity function which is typed as follows:

$$\frac{\frac{\frac{}{x : \alpha \vdash x : \alpha}}{\vdash \lambda x. x : \alpha \rightarrow \alpha}}{\vdash \lambda x. x : \Pi \alpha. \alpha \rightarrow \alpha} \quad \frac{\frac{\Gamma \vdash id : \Pi \alpha. \alpha \rightarrow \alpha}{\Gamma \vdash id : (\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha}}{\Gamma \vdash id id : \alpha \rightarrow \alpha} \quad \frac{\frac{\Gamma \vdash id : \Pi \alpha. \alpha \rightarrow \alpha}{\Gamma \vdash id : \alpha \rightarrow \alpha}}{\Gamma \vdash id id : \alpha \rightarrow \alpha}$$

$$\vdash \text{let } id = \lambda x. x \text{ in } id id : \alpha \rightarrow \alpha$$

9.0.2 Semantics

First of all we have to define the semantics of types. Since types can be open with type variables a type needs to be a mapping from a type environment to the set of types. A type environment is itself a map from the type variables to their set. However, as there is no set of sets we have to define a set containing only those sets are *small*. We call this set the *universe* of small sets and we denoted by $\mathcal{U}$. This set contains the singleton set and the set of natural numbers and it is closed under exponentials. To do this we define $\mathcal{U}_0 = \{1, \mathbb{N}\}$ and $\mathcal{U}_{n+1} = \{X^Y \mid X, Y \in \mathcal{U}_n\} \cup \mathcal{U}_n$ then the universe of monotypes is defined as $\mathcal{U} = \bigcup_{n \in \omega} \mathcal{U}_n$. We denote by $\mathcal{U}^n$ the n -fold product

$$\mathcal{U}^n \triangleq \underbrace{\mathcal{U} \times \dots \times \mathcal{U}}_{n \text{ times}}$$

Now the interpretation of a monotype A with free type variables $\text{Ftv}(A)$ is a mapping $\llbracket A \rrbracket : \mathcal{U}^{|\text{Ftv}(A)|} \rightarrow \mathcal{U}$ where $|\text{Ftv}(A)|$ is the number of free type variables in the monotype A . Thus given a semantic type environment $\iota \in \mathcal{U}^{|\text{Ftv}(A)|}$ taking type variables to semantic monotypes we can define the semantics of types by induction on the as follows:

$$\begin{aligned} \llbracket \alpha \rrbracket_\iota &= \iota(\alpha) \\ \llbracket \text{unit} \rrbracket_\iota &= 1 \\ \llbracket \text{nat} \rrbracket_\iota &= \mathbb{N} \\ \llbracket A \times B \rrbracket_\iota &= \llbracket A \rrbracket_\iota \times \llbracket B \rrbracket_\iota \\ \llbracket A \rightarrow B \rrbracket_\iota &= \llbracket B \rrbracket_\iota^{\llbracket A \rrbracket_\iota} \end{aligned}$$

The semantics of polytypes are interpreted in a bigger universe such that $\mathcal{U}$ is contained in it. The category of sets would do as opposed to the Von Neumann universe used in Gunter's book [3]. We denote the inclusion functor by $J : \mathcal{U} \hookrightarrow \mathbf{Set}$. Now for a polytype τ as a map $\llbracket \tau \rrbracket : \mathcal{U}^{|\text{Ftv}(\tau)|} \rightarrow \mathbf{Set}$. Specifically, we interpret the universal quantifier as the limit over the monotypes in $\mathcal{U}$. This is a Π -type since the universe $\mathcal{U}$ is a set with no arrows and therefore can be regarded as a discrete category as we have shown in Section 6.

$$\llbracket \Pi \alpha. \tau \rrbracket_\iota = \Pi_{X \in \mathcal{U}} \llbracket \tau \rrbracket_{\iota[\alpha \mapsto X]}$$

Recall that the Π -type is right adjoint to the diagonal functor, in this instance the base category is $\mathbf{Set}^{\mathcal{U}^n}$ with $n + 1$ being the free type variables in τ .

$$\mathbf{Set}^{\mathcal{U}^n \mathcal{U}} \xleftarrow[\Pi]{\Delta} \mathbf{Set}^{\mathcal{U}^n}$$

Notice that $\llbracket \tau \rrbracket$ lives in the category $\mathbf{Set}^{\mathcal{U}^n \mathcal{U}}$ which is the same as the category $\mathbf{Set}^{\mathcal{U}^{n+1}}$ of semantic types on $n + 1$ variables.

The isomorphism induced by the adjunction is therefore as follows:

$$\llbracket \cdot \rrbracket : \mathbf{Set}^{\mathcal{U}^{n+1}}(\Delta\Gamma, \tau) \cong \mathbf{Set}^{\mathcal{U}^n}(\Gamma, \Pi\tau) : \lceil \cdot \rceil \quad (16)$$

Notice that the homset in the category $\mathbf{Set}^{\mathcal{U}^n}$ is the set of natural transformations between functors of type $\mathcal{U}^n \rightarrow \mathbf{Set}$.

We now have to define the semantics of a context for term variables $\Gamma \equiv x_1 : \tau_1, \dots, x_n : \tau_n$. Assume m is the number of free type variables in Γ . Then $\llbracket \Gamma \rrbracket$ is a object in $\mathbf{Set}^{\mathcal{U}^m}$:

$$\llbracket \Gamma \rrbracket = \llbracket T_1 \rrbracket \times \dots \times \llbracket T_n \rrbracket$$

where the product is the defined point-wise as $(X \times Y)(\iota) = X_\iota \times Y_\iota$ for a type environment $\iota \in \mathcal{U}^m$.

The interpretation of well-typed terms $\Gamma \vdash t : \tau$ is an arrow $\llbracket \Gamma \vdash t : \tau \rrbracket : \llbracket \Gamma \rrbracket \xrightarrow{\llbracket \tau \rrbracket} \llbracket \tau \rrbracket$ in $\mathbf{Set}^{\mathcal{U}^n}$ whereas a well-typed term $\Gamma \vdash t : A$ is an arrow $\llbracket \Gamma \vdash t : A \rrbracket : \llbracket \Gamma \rrbracket \xrightarrow{\llbracket A \rrbracket} \llbracket A \rrbracket$. The interpretation is then given by induction on the typing judgment:

$$\begin{aligned} \llbracket \Gamma \vdash () : \text{unit} \rrbracket &= ! \\ \llbracket \Gamma \vdash \underline{n} : \text{nat} \rrbracket &= n \\ \llbracket \Gamma \vdash x : A \rrbracket &= \pi_x \\ \llbracket \Gamma \vdash \text{prj}_1(t) : A \rrbracket &= \pi_1 \circ \llbracket t \rrbracket \\ \llbracket \Gamma \vdash \text{prj}_2(t) : B \rrbracket &= \pi_2 \circ \llbracket t \rrbracket \\ \llbracket \Gamma \vdash (t_1, t_2) : A \times B \rrbracket &= \langle \llbracket t_1 \rrbracket, \llbracket t_2 \rrbracket \rangle \\ \llbracket \Gamma \vdash \lambda x. t : A \rightarrow B \rrbracket &= \Lambda \llbracket t \rrbracket \\ \llbracket \Gamma \vdash t_1 t_2 : B \rrbracket &= \epsilon \circ \langle \llbracket M \rrbracket, \llbracket N \rrbracket \rangle \\ \llbracket \Gamma \vdash \text{let } x = t_1 \text{ in } t_2 : A \rrbracket &= \llbracket t_2 \rrbracket \circ \langle \text{id}_\Gamma, \llbracket t_1 \rrbracket \rangle \\ \llbracket \Gamma \vdash \Lambda \alpha. t : \forall \alpha. \tau \rrbracket &= \llbracket \llbracket t \rrbracket \rrbracket \\ \llbracket \Gamma \vdash t @ A : \tau[A/\alpha] \rrbracket &= \pi_A \circ \llbracket t \rrbracket \end{aligned}$$

Most of the terms are interpreted as in Section 4. Assuming a map $\llbracket t \rrbracket : \Delta \llbracket \Gamma \rrbracket \rightarrow \llbracket \tau \rrbracket$ in $\mathbf{Set}^{\mathcal{U}^{n+1}}$ $\alpha \notin \text{Ftv}(\Gamma)$ the interpretation of the introduction rule of the $\forall$ quantifier is given by the adjunction (16) defined above which is an arrow of type

$$\llbracket \Gamma \rrbracket_\iota \rightarrow \Pi_{X \in \mathcal{U}} \llbracket \tau \rrbracket_{\iota[\alpha \mapsto X]}$$

for $\iota \in \mathcal{U}^n$.

10 The Call-By-Push-Value λ -calculus

11 Monads

In this section we are going to take a closer look at computational effects and how they can be interpreted into a category. To do this need the notion of monad. Here we assume the reader has at least heard of what a monad is from functional programming. Monads in category theory are the same concept, but originally they have been given a different definition.

Definition 11.1 (Monad). *For a category C , a monad is an endofunctor $T : C \rightarrow C$ such that there exists two natural transformations $\eta : Id \rightarrow T$ and $\mu : TT \rightarrow T$ such that the following diagrams hold:*

$$\begin{array}{ccc} T & \xrightarrow{\eta_T} & T^2 \xleftarrow{T\eta} T \\ & \searrow id_T & \downarrow \mu_T \swarrow id_T \\ & & T \end{array} \qquad \begin{array}{ccc} T^3 & \xrightarrow{\mu_T} & T^2 \\ T\mu \downarrow & & \downarrow \mu \\ T^2 & \xrightarrow{\mu} & T \end{array}$$

Example 11.1 (List Monad). *The list type $TX = 1 + A \times TX$ is a monad with $\eta_X : X \rightarrow TX$ being the map constructing a singleton list and the multiplication $\mu_X : TTX \rightarrow X$ given by concatenation applied to lists of lists.*

Example 11.2 (State Monad). *Assume a set of state S and let $T : \mathbf{Set} \rightarrow \mathbf{Set}$ be the state monad $TX = S \rightarrow X \times S$, that is the monad of computations that take a state $\sigma \in S$ and returns an output in A along with the modified state. The unit of the monad is given by $a \mapsto \lambda\sigma. \langle a, \sigma \rangle$ and the multiplication is given by*

$$c \mapsto \lambda\sigma. c'(\sigma') \text{ where } (c', \sigma') = c(\sigma)$$

For the reader more familiar with functional programming, the multiplication gives raise to the bind operation $\text{bind}_A : TA \rightarrow (A \rightarrow TB) \rightarrow TB$ defined by $\mu_A \circ T(f)$.

11.0.1 Adjunctions determine Monads

Given a pair of adjoint functors $L \vdash R$ we can construct both a monad, given by RL and a comonad, given by LR . The unit of the monad and the counit of the comonad are defined as follows

$$\begin{aligned} \eta_A &= [id_{LA}] \\ \epsilon_B &= [id_{RB}] \end{aligned}$$

The join or multiplication of the monad $\mu : RLRL \rightarrow RL$ is defined as $\mu = R\epsilon_L$ and the cojoin or comultiplication $\delta : LR \rightarrow LRLR$ is defined as $\delta = L\eta_R$. The operations of the comonad are dually defined.

11.1 The Kleisli Category

The Kleisli category is the category of arrows that produce an effect T .

Definition 11.2 (Kleisli Category). *For a category C and a monad (T, η, μ) the Kleisli category, denoted by C_T , is the category where objects are the objects in C and arrows $f : A \rightarrow B$ are arrows $f_T : A \rightarrow TB$ in C .*

We would like to stress that when we speak about arrows $f : A \rightarrow B$ in the Kleisli category C_T we mean arrows $f_T : A \rightarrow TB$ in C . We have to prove that C_T is a category. This is easy to check as for every object A we have an identity arrow $id_A : A \rightarrow A$ given by the unit of the monad $\eta_A : A \rightarrow TA$ and for arrows $f : A \rightarrow B$ and $g : B \rightarrow C$ in C_T we can construct the composite $g \circ f : A \rightarrow C$ using the functorial action of the monad $T(g)$ and the multiplication of the monad μ_C :

$$A \xrightarrow{f} TB \xrightarrow{T(g)} T^2C \xrightarrow{\mu_C} TC$$

$(g \circ f)_T$

11.2 The Computational λ -calculus

The computational λ -calculus which we denote by λ_C is a calculus with effects first devised by Eugenio Moggi [6] who was the first to discover the connection between computational effects and monads.

In this section we define the computational λ -calculus and we give it an interpretation in the Kleisli category C_T for a (strong) monad T .

11.2.1 Syntax

We first define the syntax of λ_C by defining the set of types, the terms and the typing system as usual. To demonstrate the utility of monads we are only going to need basic types and function spaces:

$$\begin{array}{lll} A, B \in \text{Types} & ::= & \text{unit} \quad (\text{unit type}) \\ & | & A \rightarrow B \quad (\text{functions}) \end{array}$$

while the set of λ -terms Terms is inductively defined as follows

$$\begin{array}{lll} t \in \text{Terms} & ::= & x \quad (\text{terms variables}) \\ & | & \text{bang} \quad (\text{effects}) \\ & | & \lambda x. t \mid t_1 t_2 \quad (\text{functions}) \end{array}$$

where bang is a program producing an effect. The typing judgement relation $\vdash$ inductively as follows

$$\frac{(x : A) \in \Gamma}{\Gamma \vdash x : A} \quad \frac{}{\Gamma \vdash \text{bang} : \text{unit}}$$

$$\frac{\Gamma \vdash t_1 : A \rightarrow B \quad \Gamma \vdash t_2 : A}{\Gamma \vdash t_1 t_2 : B} \quad \frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x. t : A \rightarrow B}$$

Note that the program bang returns nothing so we type it with the type unit.

11.2.2 Semantics

We interpret the context Γ as usual:

$$\llbracket x_1 : A_1, \dots, x_n : A_n \rrbracket = \llbracket A_1 \rrbracket \times \dots \times \llbracket A_n \rrbracket$$

The interpretation of types is then a function $\llbracket \cdot \rrbracket : \text{Types} \rightarrow \text{Obj}(C_T)$. Notice that $\text{Obj}(C_T)$ is exactly $\text{Obj}(C)$.

$$\begin{aligned} \llbracket \text{unit} \rrbracket &= 1 \\ \llbracket A \rightarrow B \rrbracket &= T\llbracket B \rrbracket^{\llbracket A \rrbracket} \end{aligned}$$

The placement of which types can produce an effect is crucial. When dealing with effects it is customary to prefer a call-by-value semantics to avoid that effectful computation passed onto functions as inputs could be executed more than once to the nature of call-by-name.

Because in call-by-value effects are computed before the β -reduction rule applies there is no need for input values to be computations, they are actually normalised values. However, once an input value is applied to a function, this can produce an effect.

We now interpret the terms of the language by induction on the typing judgment. For a well-typed term $\Gamma \vdash t : A$ we give an arrow $\llbracket t \rrbracket$ of type $\llbracket \Gamma \rrbracket \rightarrow \llbracket A \rrbracket$ in C_T as usual, but here the reader should keep in mind that this is really a map of type $\llbracket \Gamma \rrbracket \rightarrow T\llbracket A \rrbracket$ in C

$$\begin{aligned} \llbracket \Gamma \vdash () : \text{unit} \rrbracket &= ! \\ \llbracket \Gamma \vdash \text{bang} : \text{unit} \rrbracket &= b \\ \llbracket \Gamma \vdash x : A \rrbracket &= \eta_{\llbracket A \rrbracket} \circ \pi_x \\ \llbracket \Gamma \vdash \lambda x. t : A \rightarrow B \rrbracket &= \eta_{T\llbracket A \rrbracket^{\llbracket B \rrbracket}} \circ \Lambda[\llbracket t \rrbracket] \\ \llbracket \Gamma \vdash t_1 t_2 : B \rrbracket &= \text{app}(\llbracket t_1 \rrbracket, \llbracket t_2 \rrbracket) \end{aligned}$$

In order to explain the interpretation we work in the category C so the application of the monad T is explicit. We also remove the semantic brackets for the sake of removing clutter. Now, for λ -abstraction we work as follows. Assume a map $t : \Gamma \times A \rightarrow TB$. We have to define a map $\Gamma \rightarrow T(TB^A)$. By currying t we obtain $\Lambda t : \Gamma TB^A$ and by post-composing this map with η_{TB^A} we obtain the type $T(TB^A)$.

Function application is more tricky in that this is the point where we need the monad to be strong. For two maps $t_1 : \Gamma \rightarrow T(TB^A)$ and $t_2 : \Gamma \rightarrow TA$ We define the map $\text{app}(t_1, t_2) : \Gamma \rightarrow TB$ as follows:

$$\begin{array}{ccc} \Gamma & \xrightarrow{\quad \text{app}(t_1, t_2) \quad} & TB \\ \downarrow \langle t_1, t_2 \rangle & & \uparrow \mu_B \\ T(TB^A) \times TA & & T^2 B \\ \downarrow s_{TB^A, TA} & & \uparrow \mu_{TB} \\ T(TB^A \times TA) & \xrightarrow{T(s_{TB^A, A})} T(T(TB^A \times A)) & \xrightarrow{T^2(\epsilon)} T^3 B \end{array}$$

Exercise 11.1. Let T be the free monoids monad in **Set**, i.e. the monad that maps a set A to the set of finite lists of elements of A . Show that the category of algebras for T is isomorphic to the category of monoids.

References

- [1] Steve Awodey. *Category Theory*. Oxford University Press, Inc., USA, 2nd edition, 2010.
- [2] Jean-Yves Girard. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. Thèse d'État, Université Paris VII, 1972.
- [3] C.A. Gunter. *Semantics of Programming Languages: Structures and Techniques*. Foundations of computing. MIT Press, 1992.
- [4] Saunders MacLane. *Categories for the Working Mathematician*. Springer-Verlag, New York, 1971. Graduate Texts in Mathematics, Vol. 5.
- [5] B. Milewski. *Category Theory for Programmers*. Blurb, Incorporated, 2018.
- [6] Eugenio Moggi. Notions of computation and monads. *Inf. Comput.*, 93(1):55–92, 1991.
- [7] Andrew M. Pitts. Polymorphism is set theoretic, constructively. In David H. Pitt, Axel Poigné, and David E. Rydeheard, editors, *Category Theory and Computer Science, Edinburgh, UK, September 7-9, 1987, Proceedings*, volume 283 of *Lecture Notes in Computer Science*, pages 12–39. Springer, 1987.
- [8] John C. Reynolds. Types, abstraction and parametric polymorphism. In R. E. A. Mason, editor, *Information Processing 83, Proceedings of the IFIP 9th World Computer Congress, Paris, France, September 19-23, 1983*, pages 513–523. North-Holland/IFIP, 1983.
- [9] John C. Reynolds. Polymorphism is not set-theoretic. In Gilles Kahn, David B. MacQueen, and Gordon D. Plotkin, editors, *Semantics of Data Types, International Symposium, Sophia-Antipolis, France, June 27-29, 1984, Proceedings*, volume 173 of *Lecture Notes in Computer Science*, pages 145–156. Springer, 1984.
- [10] Philip Wadler. Theorems for free! In Joseph E. Stoy, editor, *Proceedings of the fourth international conference on Functional programming languages and computer architecture, FPCA 1989, London, UK, September 11-13, 1989*, pages 347–359. ACM, 1989.